

Diseño Arquitectónico y Funcional

Introducción

El objetivo de esta etapa es definir y estructurar el diseño del sistema a nivel de arquitectura, modelo de datos, experiencia de usuario e interfaz gráfica, utilizando como insumo los requerimientos identificados en la fase anterior. A partir de estos insumos se establecen los criterios de diseño, se documentan las decisiones arquitectónicas y se elaboran las diferentes vistas y diagramas que permiten comprender de manera integral la solución que se implementará en las siguientes etapas. Adicionalmente, la propuesta visual del sistema se materializa mediante prototipos de baja fidelidad, los cuales facilitan la validación temprana con los usuarios y permiten incorporar retroalimentación antes de iniciar el desarrollo e implementación formal.

Diseño Arquitectónico

Introducción al enfoque ADD

El Attribute-Driven Design (ADD) es un método de diseño arquitectónico que se centra en los atributos de calidad como principales motores del diseño, en contraste con enfoques que inician desde componentes técnicos o patrones predeterminados. Así, ADD comienza por identificar los drivers arquitectónicos, es decir, aquellos requerimientos funcionales y no funcionales que tienen mayor impacto estructural, y los utiliza como criterio para descomponer el sistema en módulos coherentes.

Para este proyecto se escogió ADD motivado en que:

- El sistema tendrá múltiples perfiles de usuario.
- Se pretende integrar servicios externos (pagos, validaciones, notificaciones).
- Existen requisitos regulatorios y de protección de datos.
- Se espera evolucionar el producto sin comprometer su mantenibilidad.

Por tanto, adoptar ADD permite:

- Reducir el riesgo de decisiones tecnológicas tempranas no justificadas.
- Alinear arquitectura con objetivos de negocio.
- Garantizar trazabilidad entre requerimientos y decisiones estructurales.
- Diseñar con intención explícita sobre calidad (no como consecuencia accidental).

Identificación de Stakeholders y Contexto de Diseño

En pro de seguir la línea de un diseño centrado en el usuario, ADD también promueve mapear quién se ve afectado por las decisiones que se toman, de este modo surgen los siguientes stakeholders:

- **primarios:** arrendadores, arrendatarios.
- **secundarios:** entidades financieras, proveedores de firma electrónica, servicios de validación.
- **regulatorios:** cumplimiento de protección de datos (Habeas Data en Colombia).

Elección de los Drivers Arquitectónicos

Para esto se tomó como insumo de la sección anterior:

- Requerimientos funcionales críticos (casos de uso core).
- Requerimientos no funcionales priorizados.
- Riesgos técnicos identificados.
- Restricciones del entorno (regulatorias, presupuestales, tecnológicas).

Es importante hacer la claridad en que no todos los atributos de calidad son drivers arquitectónicamente relevantes, así que se definen los siguientes criterios para determinar que un atributo se convierte en driver arquitectónico cuando:

- Tiene alto impacto estructural.
- Introduce decisiones irreversibles o costosas de cambiar.
- Está asociado a alto riesgo si no se cumple.
- Diferencia competitivamente al producto.

Drivers explícitos

Son aquellos que podemos relacionar directamente con alguno de los requerimientos ya identificados:

Driver (ADD)	Qué significa en este producto	Requerimientos origen (traza)	Justificación (por qué es driver)
D1. Mobile-first y compatibilidad real en celular	La UX y performance se optimizan para móvil como canal principal, sin romper escritorio.	NFR-01 “Mobile-first y correcto funcionamiento en celular” + enfoque “mobile-first” en requerimientos	Si el sistema no funciona “perfecto” en celular, se rompe el canal predominante y cae la adopción.
D2. Usabilidad e inclusión digital (baja carga cognitiva + guía)	Flujos progresivos, lenguaje claro, asistencia contextual, evitar saturación de decisiones/pasos.	NFR-02 “Baja carga cognitiva” y NFR-03 “Claridad explicativa” + hallazgos de preferencia por simplicidad/guía	El SRS muestra que la fricción principal no es “falta de features”, sino confusión, incertidumbre y dificultad para saber “qué sigue”, especialmente en adultos/adultos mayores.
D3. Accesibilidad y legibilidad	Contrastes, tipografías, controles claros; UX apta para limitaciones visuales	NFR-05 “Diseño accesible y legible” + requisitos de accesibilidad/consistencia visual	Si no es accesible/legible, se amplía la brecha de adopción y el sistema excluye justo al segmento crítico

	y baja alfabetización digital.		(arrendadores mayores).
D4. Tolerancia a errores y recuperación	Mensajes de error accionables, preservación de datos, reintentos controlados, estados consistentes.	NFR-04 “Retroalimentación visible ante errores” + requerimiento de recuperación sencilla ante errores	Un error sin guía genera abandono inmediato. Además, en flujos transaccionales (pagos/firma) el error percibido se interpreta como riesgo.
D5. Confianza y transparencia en procesos críticos (pagos + firma)	Confirmaciones visibles, evidencia del estado, explicaciones de validez, reducción de incertidumbre.	NFR-06 “Generar confianza en pagos y firma digital” + “refuerzo de confianza” y explicación al usuario	En tu dominio, la confianza es el “habilitador” del uso: sin ella, el usuario vuelve a WhatsApp/presencial y el valor del producto se pierde.
D6. Integración confiable con servicios externos (pagos/firma) + resiliencia	Orquestación robusta, manejo de fallos, timeouts, trazabilidad, degradación controlada.	Dependencia de servicios externos (pagos/firma) + riesgos por disponibilidad/estabilidad de APIs externas	Arquitectónicamente condiciona todo: contratos y pagos dependen de terceros; si no diseñas resiliencia, la plataforma se vuelve frágil y poco confiable.
D7. Cumplimiento legal y privacidad de datos	Protección de datos personales, tratamiento y consentimiento, y soporte a firma electrónica conforme normativa.	Restricción explícita de cumplimiento (Ley 1581/2012; normativa firma electrónica)	No es un “nice-to-have”: si se viola, el sistema no es viable (riesgo legal y reputacional). Además afecta diseño de datos, logging y seguridad.
D8. Trazabilidad documental y evidencias (auditoría funcional)	Historial de pagos, comprobantes, consulta de contratos; evidencia accesible y consistente.	“Consulta de contratos y comprobantes”, “historial y registro de pagos”	Refuerza confianza y control. Sin evidencia verificable, aumentan disputas y se degrada la percepción de seriedad del sistema.

Drivers implícitos y supuestos

Aunque los requerimientos enfatizan fuertemente usabilidad, claridad y accesibilidad, existen atributos que los usuarios dan por sentados y rara vez formulan explícitamente. De este modo, algunos atributos son expectativas implícitas del usuario, que si no se satisfacen, invalidan la experiencia incluso antes de evaluar la usabilidad o el valor funcional.

Por esta razón, además de los atributos derivados directamente del SRS, se identifican supuestos arquitectónicos de calidad que deben considerarse como drivers estructurales.

Driver (ADD)	Qué significa en este producto	Justificación (por qué es driver)
D9. Desempeño	Las acciones no generan esperas prolongadas. Los pagos y confirmaciones no tardan de forma incierta.	Si el sistema responde lentamente, la percepción de riesgo aumenta, especialmente en procesos de pago o firma digital.
D10. Disponibilidad	El sistema está disponible cuando lo necesitan y no “se cae” en momentos críticos, por ejemplo, cuando se va a pagar o consultar un contrato.	En plataformas digitales transaccionales, la indisponibilidad impacta directamente la confianza y puede llevar al abandono definitivo.
D11. Seguridad	Los datos de los usuarios no deberían estar expuestos, no deben existir cobros indebidos, y el contrato tiene validez.	La seguridad no es percibida como una funcionalidad, sino como una condición mínima de existencia.

Priorización de los drivers

Para un primer acercamiento a la arquitectura del sistema, se agrupan los drivers que están cohesionados entre sí y se priorizan según el impacto en la estrategia de negocio, riesgo si falla, costo de cambio posterior e influencia estructural en la arquitectura. Con esto en mente se determina tomar los 4 más relevantes, sin embargo esto no significa que la arquitectura pueda seguir evolucionando a medida que el producto y la madurez del equipo o la tecnología también lo vayan haciendo.

Prioridad	Driver	Naturaleza	Justificación
1	Usabilidad e Inclusión Digital	Diferenciador estratégico	Es el diferencial estratégico del producto. Los hallazgos muestran que el principal problema del mercado no es la inexistencia de plataformas, sino la fricción, confusión y

			baja apropiación digital.
2	Integración Resiliente	Dependencia estructural	El modelo de negocio depende directamente de pagos digitales y contratos con firma electrónica.
3	Seguridad y Cumplimiento	Condición de viabilidad	La plataforma gestiona información sensible (datos personales, contratos, pagos). El incumplimiento tiene impacto legal y reputacional.
4	Performance y Disponibilidad	Expectativa implícita mínima	Se agrupan porque ambos forman parte de la percepción mínima de “funciona bien”.

Escenarios de calidad

Un escenario de calidad describe de forma cuantificable y verificable el comportamiento del sistema bajo condiciones específicas por cada atributo de calidad, y se plantean usando los siguientes elementos:

- **Fuente:** generador u origen del evento que se va a evaluar
- **Estímulo:** el evento concreto que afecta el sistema
- **Artefacto:** el sistema o componentes del mismo
- **Entorno:** las condiciones operativas en las que se va a evaluar el comportamiento del sistema
- **Respuesta:** Accionarial del sistema ante el estímulo
- **Medida:** métrica para validar el éxito del sistema frente a el estímulo en las condiciones operativos definidas.

Con esto en mente, un escenario permiten tener un panorama de cómo se espera que se comporte el sistema en diferentes condiciones para cada atributo de calidad, lo que se puede entender cómo objetivos de la arquitectura y por tanto insumos para identificar qué patrones y tácticas pueden aplicarse para lograr cumplir con ellos. Así, se utiliza esta técnica para remover la subjetividad de los atributos y enriquecer el proceso de diseño.

Fuente	Estímulo	Artefacto	Entorno	Respuesta	Medida
Usabilidad e Inclusión Digital (UX + Accesibilidad)					

Usuario con baja apropiación digital	Inicia el proceso de crear una publicación	UI + Backend	Operación normal	El sistema guía por pasos secuenciales, valida en tiempo real y confirma progreso	≥ 80% de usuarios completan el flujo sin abandonar Tiempo promedio de finalización ≤ 5 minutos ≤ 1 decisión principal por pantalla
Usuario general	Ingresa dato inválido en formulario	UI + validación	Operación normal	Se muestra mensaje claro con ejemplo y foco en campo	100% de validaciones con mensaje accionable Corrección del error ≤ 30 segundos Tasa de abandono por error ≤ 5%
Usuario con limitación visual leve	Navega y ejecuta acción principal	UI	Operación normal	Componentes accesibles y legibles	Cumplimiento WCAG 2.1 AA ≥ 95% Contraste ≥ 4.5:1 Área táctil ≥ 44px
Integración Resiliente con Servicios Externos					

Servicio externo (pagos/firma)	Timeout > 5 segundo	Backend	Operación normal	Se activa timeout controlado, estado pasa a “PENDING”, usuario recibe mensaje claro	Tiempo de respuesta al usuario ≤ 2 segundos. Reintento automático máximo 2 veces con backoff exponencial.
Servicio externo	Indisponibilidad durante 10 minutos	Backend	Hora pico	Sistema degrada funcionalidad dependiente sin afectar otras operaciones	Disponibilidad general del sistema ≥ 99.5% 0 caída completa de la plataforma
Usuario general	Reintenta operación de pago tras error visual	Backend	Operación normal	No se genera doble cobro	0 transacciones duplicadas Idempotency-key validada en 100% de operaciones críticas
Seguridad y Cumplimiento Legal					
Usuario autenticado	Intenta acceder a contrato de otra propiedad	Backend	Operación normal	Se bloquea con 403 y se registra auditoría	100% de accesos no autorizados bloqueados
Evento interno	Almacenamiento de información de identificación personal	Base de datos	Operación normal	Datos sensibles cifrados en reposo	100% de campos PII cifrados TLS 1.2+ obligatorio en tránsito 0 exposición de PII en logs

Usuario	Firma contrato	Backend + almacenamiento documental	Operación normal	Se registra evento con timestamp y hash verificable	100% de contratos con hash verificable
Performance y Disponibilidad					
Usuario	Consulta listado de inmuebles	Backend	Operación normal	Devuelve respuesta rápida y consistente	Tiempo de respuesta ≤ 800ms Error rate < 1%
Usuario	Abre página principal	UI	Dispositivo móvil, Red 4G promedio	Interfaz usable rápidamente	LCP ≤ 2.5s
Sistema completo	Caída de nodo backend	Sistema completo	Operación normal	Servicio accesible	Disponibilidad ≥ 99.5% MTTR ≤ 30 minutos

Restricciones y Alcance Arquitectónico Inicial

En el proceso de diseño arquitectónico es importante no solo definir los atributos de calidad sino también las restricciones y el alcance que acotan las soluciones que pueden satisfacer los mismos drivers. De este modo, dado que el sistema será desarrollado en una etapa inicial como **prototipo con características similares a un MVP**, los lineamientos aquí definidos buscan equilibrar **baja complejidad, bajo costo y una arquitectura de calidad**, garantizando bases sólidas para su futura evolución.

1. Enfoques Arquitectónicos:

- Prototipo o Versión MVP Arquitectónica:**

La versión inicial del sistema se diseñará con un alcance reducido, centrado en la validación de los requerimientos esenciales y la viabilidad técnica. Se priorizará **la simplicidad, el bajo costo** y la **entregabilidad rápida**, manteniendo estándares mínimos de **cohesión, modularidad y mantenibilidad**.
- Arquitectura Evolutiva o To-Be:**

A medida que el producto alcance madurez y cuente con mayor presupuesto, se evolucionará hacia una arquitectura **más desacoplada y escalable**, reforzando aspectos de rendimiento, seguridad y disponibilidad. El diseño inicial debe establecer una base flexible que permita dicha transición sin comprometer la integridad del sistema.

2. Restricciones y Supuestos:

- El alcance funcional se limita a los requerimientos críticos validados durante la fase de recolección y análisis.

- Las decisiones tecnológicas iniciales serán revisables, priorizando interfaces bien definidas que faciliten cambios futuros.
- El prototipo podrá implementarse en entornos locales o de bajo costo (e.g., contenedores o servicios cloud gratuitos) con miras a su posterior escalamiento.
- Los atributos de calidad se abordarán progresivamente, enfatizando mantenibilidad y simplicidad en las primeras fases.
- Se prioriza la coherencia del diseño sobre la completitud técnica inicial.

3. Estrategia de Evolución:

- Toda decisión arquitectónica deberá ser trazable hacia *drivers* explícitos o implícitos documentados en esta etapa.
- Se documentarán las diferencias entre la arquitectura *as-is* (MVP) y la *to-be*, identificando qué componentes pueden ser reutilizados, reemplazados o escalados.

Vista de Contexto

Esta sección describe la vista de contexto del sistema siguiendo el Modelo C4 (nivel 1), conectando los actores y sistemas externos identificados en el Documento de Especificación de Requerimientos de Software con la plataforma a diseñar. El objetivo es ofrecer una visión de alto nivel que sea comprensible para stakeholders técnicos y no técnicos, manteniendo trazabilidad con los drivers arquitectónicos D1–D11 y con las historias de usuario del backlog (por ejemplo US-01, US-02, US-06, US-10, US-14, US-18).

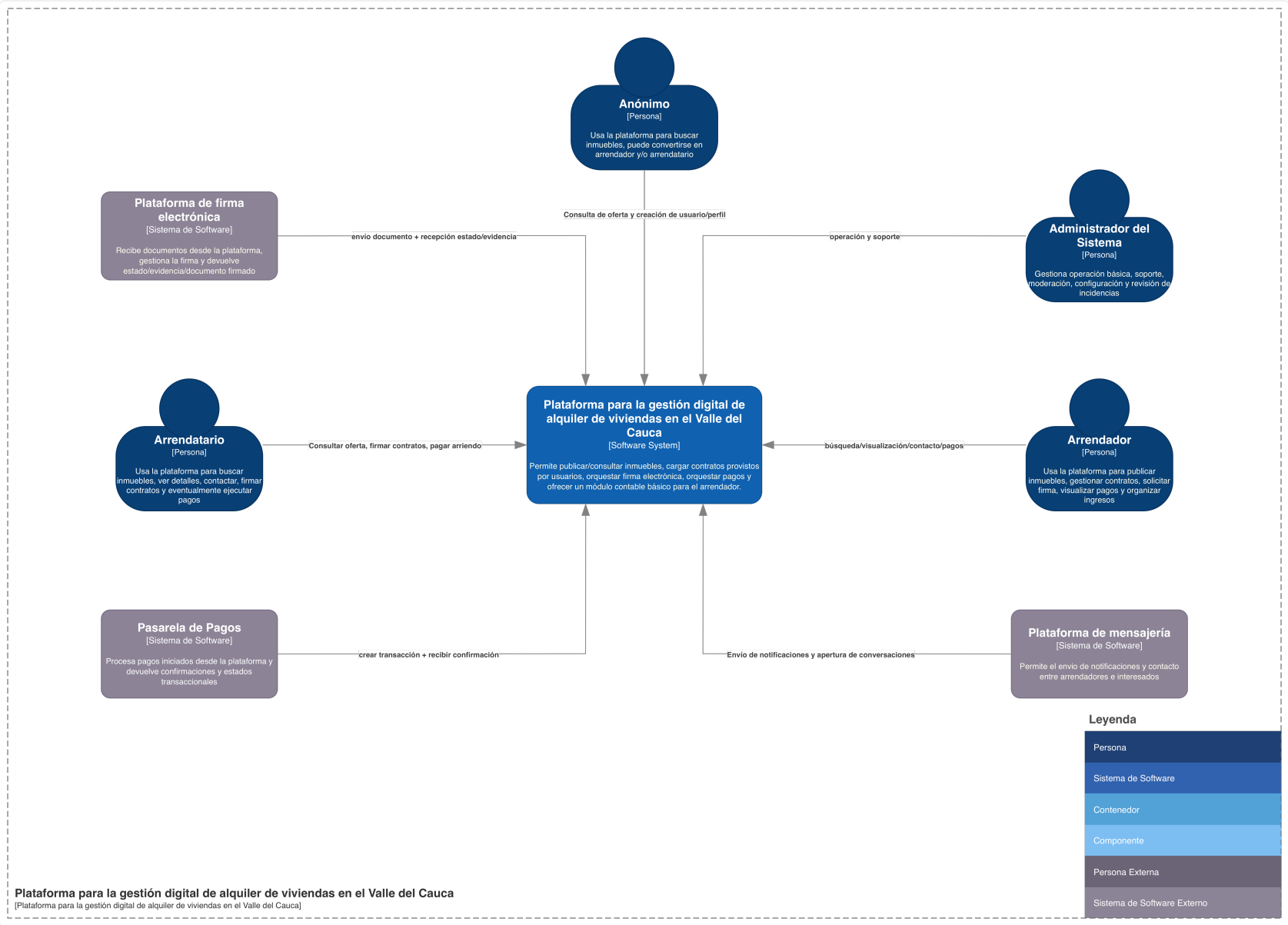
Elementos principales de la vista de contexto

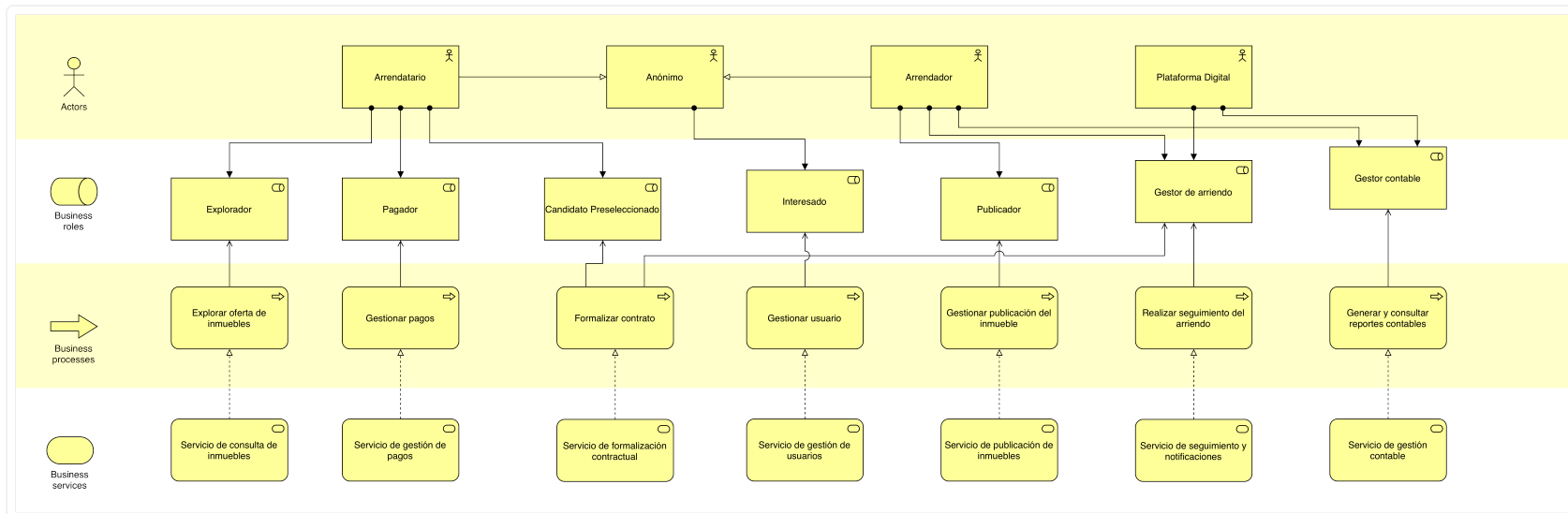
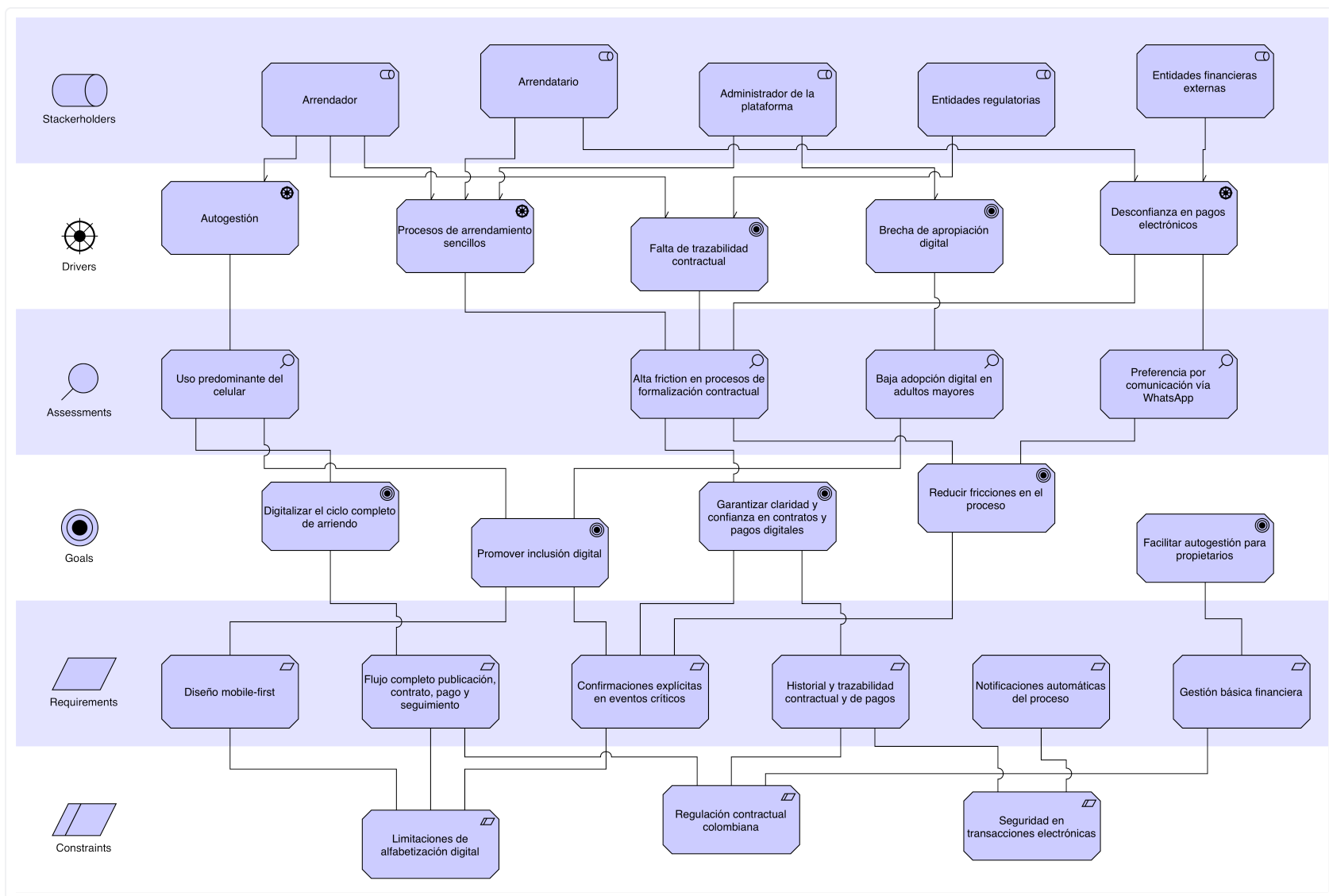
- **Sistema principal:** Plataforma de gestión de arriendo de vivienda urbana para el Valle del Cauca, que actúa como orquestador del ciclo de arriendo (publicación de inmuebles, exploración de oferta, formalización contractual y gestión básica de pagos), apoyándose en servicios externos especializados.
- **Actores humanos:**
 - **Arrendador:** persona (frecuentemente adulto o adulto mayor) que publica y administra uno o varios inmuebles en arriendo. Interactúa con la Plataforma principalmente para crear y actualizar publicaciones (US-06, US-07, US-08), gestionar contratos (US-10, US-12) y consultar pagos y reportes (US-15, US-16, US-17).
 - **Arrendatario:** persona (frecuentemente joven o adulto) que explora la oferta de inmuebles, contacta a arrendadores y participa en la firma de contratos y realización de pagos (US-01, US-02, US-03, US-04, US-05, US-11, US-12, US-14, US-18).
 - **Administrador de la Plataforma (rol operativo/técnico):** usuario interno encargado de configuración básica, monitoreo y soporte de primer nivel, especialmente en etapas tempranas del prototipo.
 - **Usuario Anónimo:** persona que puede consultar la oferta de inmuebles sin necesidad de autenticación. Para realizar cualquier funcionalidad adicional, debe crear un perfil y convertirse en arrendador y/o arrendatario.
 - Desplazarse para ver más.
- **Sistemas externos clave:**

- **Servicio de firma electrónica:** proveedor externo que permite la firma digital con validez jurídica de los contratos de arrendamiento (US-12, NFR-06), sujeto a la normativa de firma electrónica vigente en Colombia.
- **Pasarela de pagos:** servicio externo que procesa pagos electrónicos del canon de arrendamiento (US-14, US-15, US-16), manejando medios como PSE, tarjetas débito/crédito u otros mecanismos digitales aceptados.
- **Canal de mensajería (ej. WhatsApp):** canal preferente para notificaciones y comunicación entre arrendador y arrendatario, cuando sea pertinente y conforme a las políticas de uso del sistema (US-05, US-19, US-23).

Relaciones de interacción (alto nivel)

- **Arrendatario → Plataforma:** consulta la oferta sin autenticación obligatoria, aplica filtros básicos (zona, precio, características), visualiza detalles y fotos de los inmuebles, administra listas de interés y eventualmente inicia contacto con el arrendador. Esta relación está guiada por los drivers D1 (mobile-first), D2 (usabilidad e inclusión digital), D3 (accesibilidad) y D5 (confianza y transparencia).
- **Arrendador → Plataforma:** publica inmuebles con fotos obligatorias, recibe notificaciones de interesados, gestiona contratos y monitorea pagos y reportes simples. Esta interacción se ve influenciada por D1, D2, D5, D7 (cumplimiento legal y privacidad) y D8 (trazabilidad documental y evidencias).
- **Plataforma → Servicio de firma electrónica:** orquesta el envío de contratos para firma, recibe el resultado del proceso (éxito, error, estado intermedio) y actualiza el estado del arriendo. Aquí son críticos D5 (confianza en firma), D6 (integración confiable) y D7 (cumplimiento legal).
- **Plataforma → Pasarela de pagos:** inicia y monitorea transacciones de pago, registra comprobantes y estados, y expone al usuario un historial entendible y verificable. Esta relación está dominada por D5 (confianza en pagos), D6 (resiliencia ante fallos de terceros) y D8 (evidencias e historia de pagos).
- **Plataforma → Canal de mensajería (WhatsApp u otros):** utiliza canales familiares para enviar notificaciones relevantes del proceso (por ejemplo, nuevo interesado, contrato disponible para firma, pago recibido), siempre respetando la privacidad y consentimiento del usuario (D2, D5, D7).
- **Anónimo → Plataforma:** consulta la oferta sin autenticación obligatoria, pero debe crear un perfil para desbloquear el resto de funcionalidades como arrendador o arrendatario.





Iteraciones para el diseño de la arquitectura

En el enfoque Attribute-Driven Design (ADD), la arquitectura no se define de una sola vez, sino mediante iteraciones sucesivas de diseño, cada una guiada por los drivers arquitectónicos priorizados. En cada iteración se realizan los siguientes pasos:

1. **Establecer el objetivo y los drivers arquitectónicos** que guiarán la iteración (atributos de calidad, requerimientos críticos o restricciones de alto impacto).
2. **Elegir el elemento arquitectónico a descomponer**, comenzando normalmente por el sistema completo en la primera iteración.

- 3. **Escoger conceptos de diseño** como arquitecturas de referencia, patrones y tácticas que permitan satisfacer el objetivo y los drivers seleccionados.
- 4. **Identificar las responsabilidades** que dicho elemento debe cumplir, **proponer una descomposición en sub elementos** que permitan satisfacer esas responsabilidades, y **definir interfaces y mecanismos de interacción** entre los sub elementos.
- 5. **Crear vistas y documentar las decisiones arquitectónicas.**
- 6. **Analizar el diseño resultante**, posibles trade-offs y verificar que la descomposición satisface el objetivo y los drivers seleccionados.
- 7. **Repetir el ciclo sobre los nuevos sub elementos**, refinando progresivamente la arquitectura hasta alcanzar el nivel de detalle necesario para implementación.

Iteración 1

Objetivo
Definir la estructura del sistema
Drivers

- Bajo costo/costo eficiencia*
- Simplicidad*
- Adaptabilidad (debe funcionar tanto en escritorio como celular)
- Interoperabilidad
- Usabilidad

*Restricciones

Elemento a descomponer
Plataforma de gestión de arriendo de vivienda urbana

Conceptos de diseño
Para esta primer iteración, al tratarse de definir el sistema como un todo, se tomaran como conceptos de diseño, principalmente estilos arquitectónicos, así:

Estilo Arquitectónico	Descripción	Beneficios	Trade-offs
Monolito Modular	Se trata de una arquitectura monolítica, es decir, una sola unidad de software, sin embargo, la lógica se divide entre dominios de negocio bien definidos y limitados.	Alta simplicidad; Bajo costo de implementación; Si se implementa correctamente puede tener un costo bajo de mantenibilidad.	Baja elasticidad y escalabilidad por su naturaleza monolítica.

Por Capas	En este estilo la organización se basa en capas lógicas en la que cada una desempeña una función específica lo que permite desarrollo en paralelo en equipos dedicados a cada capa. Existen variaciones para las unidades de software implementadas.	Alta simplicidad; Bajo costo de implementación, debido a que tradicionalmente los desarrolladores están familiarizados con ella.	Baja elasticidad y escalabilidad.
Basada en eventos	Estilo distribuido con comunicación asíncrona basada en la gestión de eventos	Alta modularidad; Alto rendimiento; Alta escalabilidad.	Baja simplicidad; Alta complejidad en pruebas.
Microservicios	Estilo basado en unidades autosuficientes de contextos acotados que permiten desacoplar y paralelizar el trabajo.	Alta modularidad; Alta escalabilidad.	Baja simplicidad; Rendimiento medio si involucra muchos llamados a través de la red.

Teniendo en cuenta los drivers priorizados para esta primera iteración, los estilos arquitectónicos que ofrecen mayores beneficios en términos de simplicidad estructural, menor complejidad operativa y costo reducido son el **monolito modular** y la **arquitectura por capas**. Sin embargo, se selecciona el **monolito modular** como estilo base porque, aunque comparte algunos trade-offs con la arquitectura por capas, ofrece una separación explícita por módulos con límites bien definidos que facilitan la evolución progresiva del sistema. Esta decisión permite iniciar con una arquitectura adecuada para el enfoque de MVP, pero manteniendo la posibilidad de extraer módulos hacia microservicios o incorporar comunicación basada en eventos distribuidos en futuras iteraciones, sin requerir una reestructuración radical del dominio.

Sub elementos

Considerando el estilo de arquitectura seleccionado, para determinar los sub elementos y las interfaces que los comunican, primero se determina cuales dominios de negocio se van abordar como módulos del sistema:

- **Dominio de oferta de inmuebles**

- **Dominio de portafolio del arrendador**
- **Dominio de formalización contractual**
- **Dominio de pagos**
- **Dominio de contabilidad**
- **Dominio de seguimiento del proceso de arriendo**
- **Dominio de usuarios**
- **Dominio de notificaciones**

Con esto en mente los dominios que se relacionan entre sí desde el punto de vista del flujo del usuario y la estrategia de negocio :

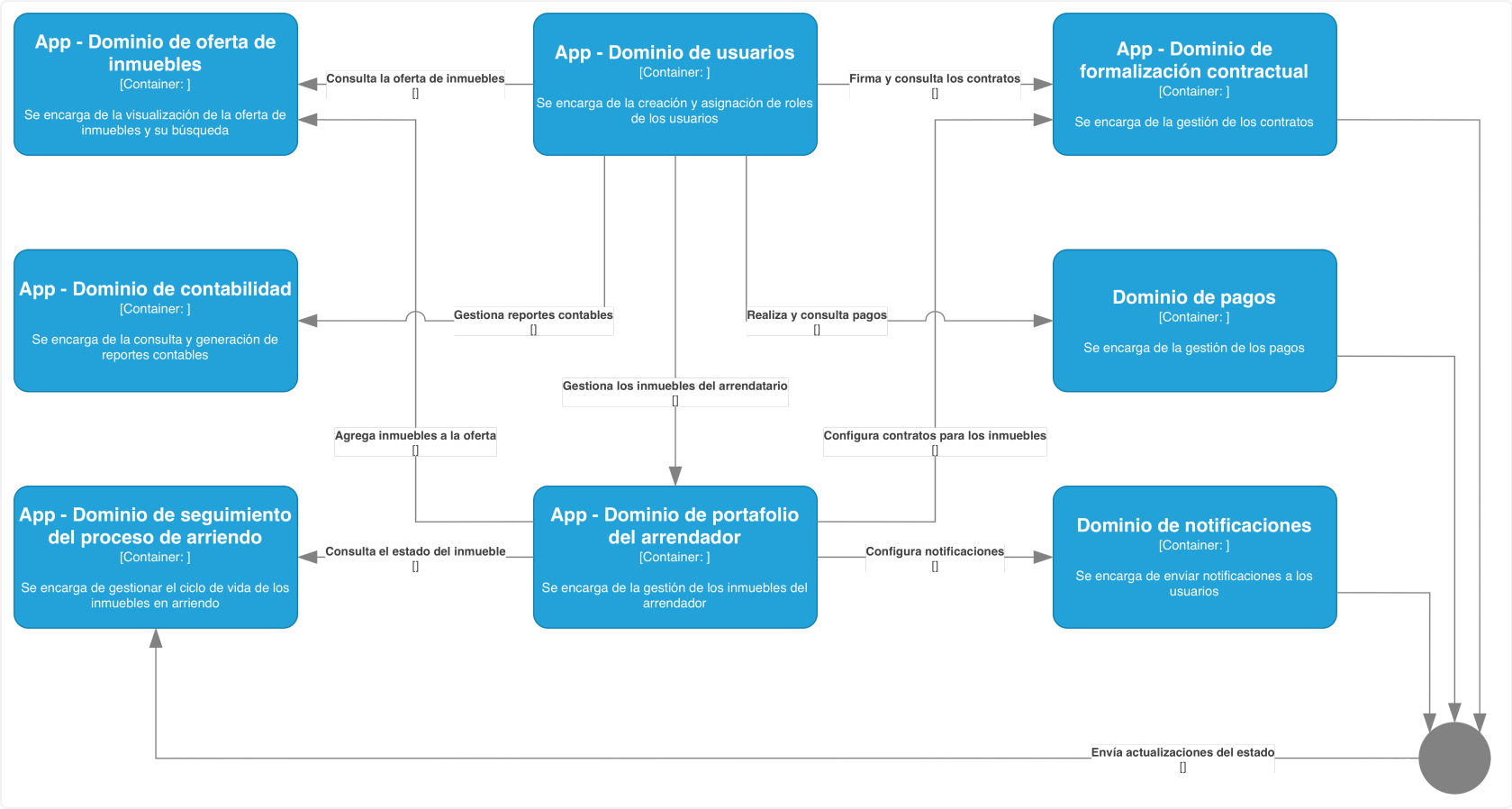
- **Dominio de usuarios** con el **dominio de portafolio del arrendador** debido a que se debe ser un usuario registrado para gestionar su portafolio de inmuebles.
- **Dominio de oferta de inmuebles** con el **dominio de portafolio del arrendador** debido a que cuando un arrendador decida agregar un inmueble a su portafolio, este puede convertirse en parte de la oferta de inmuebles.
- **Dominio de formalización contractual** con el **dominio de portafolio del arrendador** debido a que se debe tener el inmueble configurado en el portafolio para poder asignarle un contrato y gestionarlo.
- **Dominio de seguimiento del proceso de arriendo** con el **dominio de portafolio del arrendador** debido a que se debe tener el inmueble configurado en el portafolio para poder tener los estados en los que se encuentra.
- **Dominio de usuarios** con el **dominio de notificaciones** debido a que se debe tener creado un perfil para poder configurar las notificaciones.
- **Dominio de pagos** con el **dominio de contabilidad** debido a que se necesita que los pagos sean gestionados a través de la plataforma para poder generar reportes contables a partir de ellos.
- **Dominio de usuarios** con el **dominio de contabilidad** debido a que se debe tener un usuario para poder consultar los reportes de pagos.
- **Dominio de usuarios** con el **dominio de pagos** debido a que todo pago siempre está asociado a un usuario arrendatario y a un arrendador.
- **Dominio de oferta de inmuebles** con el **dominio de formalización contractual** debido a que un contrato se genera a partir de una oferta aceptada.
- **Dominio de seguimiento del proceso de arriendo** con el **dominio de notificaciones** debido a que muchos eventos de la línea de tiempo disparan notificaciones ("contrato firmado", "pago recibido", etc.).

Decisiones arquitectónicas

ID	Título	Contexto	Decisión	Consecuencias
AD-01	Estilo arquitectónico	Para la primer versión del sistema es clave la simplicidad y un bajo costo	Monolito Modular.	Simplicidad en implementación pero posibilidad de evolución a una arquitectura más

		pero se espera que su evolución también sea sencilla.		robusta debido a su modularidad.
--	--	---	--	----------------------------------

Vistas



Iteración 2

Objetivo

Definir la **arquitectura de aplicación** (estructura interna y reglas de interacción) para el monolito modular MVP

Drivers

- Seguridad
- Desempeño
- Disponibilidad

Elemento a descomponer

El sistema a nivel de aplicación

Conceptos de diseño

Para esta segunda iteración, al tratarse de definir una línea base de arquitecturas para los módulos de aplicación, se tomaran los siguientes conceptos de diseño:

Concepto de diseño	Descripción	Beneficios	Trade-offs
Estilos			
Diseño detallado por capas por módulo	Tradicionalmente se tienen las capas de controlador, servicio y repositorio para	Simple; Rápido de implementar; Fácil de entender.	Se puede cometer el antipatrón de implementar “God services”, reglas de

	atender el ciclo de vida de peticiones e interacción con las entidades de negocio.		seguridad dispersas, e integraciones que contaminan el dominio, lo que lo pueden hacer difícil evolucionar.
Arquitectura Hexagonal	Se aplica el patrón de puertos y adaptadores que consiste en abstraer la comunicación entre la aplicación y la infraestructura, de modo tal que se limpia la dependencia y siempre y cuando se tenga el adaptador adecuado se puede hacer uso de la lógica de negocio.	Alto desacoplamiento interno; Alta testeabilidad; Alta adaptabilidad tecnológica por su abstracción de la infraestructura.	Convivencia de estilos si no hay reglas; requiere convenciones estrictas.
Clean Architecture	Busca delimitar las responsabilidades del código al estructurarlo en capas concéntricas independientes que permitan a su vez alistar la infraestructura de la lógica de negocio de la aplicación.	Alto desacoplamiento interno y separación de responsabilidades; Alta testeabilidad; Alta adaptabilidad tecnológica por su abstracción de la infraestructura.	Alta complejidad debido al sobre diseño; Aumento en el código para generar las abstracciones y comunicación entre capas; Se puede deteriorar el rendimiento por la cantidad de capas que involucra.
Patrones			
Circuit breaker	Este patrón permite abordar las tácticas de manejo de excepciones y re intento, tiene como objetivo evitar la degradación y latencia innecesaria de la aplicación ante la falla en la	Gestiona la lógica de re intentos antes de una falla; Controla la propagación de cambios; Permite incorporar lógica de fallback.	Se puede caer en una latencia innecesaria si los tiempos de espera son muy altos; se puede indisponibilizar el servicio inadecuadamente si

	respuesta de un servicio externo.		el tiempo de espera es muy corto.
Validador interceptor	Este patrón permite interceptar las peticiones antes de que lleguen al recurso destino e implementar tácticas de verificación de la integridad del mensaje.	Puede cubrir diferentes tácticas de seguridad asociadas a la detección de ataques.	Al introducir un elemento intermedio afecta el rendimiento.
Attribute Based Access Control (ABAC)	Permite controles granulares basados en distintos atributos como el usuario, el recurso al que se intenta acceder y la operación que se quiere realizar.	Control granular; Reducción de la superficie de confianza.	Puede aumentar la complejidad de implementación y disminuir el rendimiento
Role Based Access Control (RBAC)	Permite controles de acceso basados en los roles de los usuario.	Costo relativamente bajo de implementación al agrupar permisos a usuarios basados en su rol; Simple dado que permite en gran medida mapear el comportamiento de permisos reales en el sistema.	Se debe tener cuidado con la superposición de roles para evitar conflictos o accesos indebidos.

Considerando que la arquitectura que se está definiendo es un MVP pero debe ser evolutiva se elige como estilo arquitectónico a nivel de aplicación la **arquitectura hexagonal** al brindar balance entre abstracción de la infraestructura y complejidad de implementación respecto a las otras opciones, lo que permite que eventualmente pueda evolucionar a arquitecturas más robustas como clean architecture. Respecto a la seguridad, se escoge usar **interceptores validadores** para detectar ataques y el uso de **RBAC con verificación de pertenencia de recursos**, es decir que según el rol del usuario se autoriza el acceso a características especiales de la aplicación pero adicionalmente se verifica que cada usuario solo pueda acceder a recursos que le pertenezcan.

Sub elementos

Dominios de línea base, es decir, todo el resto de dominios con casos más específicos también van a cumplir o heredar estos sub elementos de la línea base:

Arquitectura hexagonal.

- Control de accesos basados en roles.

Dominios con APIs expuestas para consumo externo al sistema:

- Patrón interceptor validador en la entrada de datos de la UI y APIs expuestas por fuera del sistema.

Dominios con APIs

- Patrón circuit breaker en los adaptadores para consumo de servicios externos.

Respecto a la comunicación entre los diferentes dominios, para garantizar un bajo acoplamiento se plantea que se realice a través de APIs. Por otro lado, considerando que se espera una aplicación que sea adaptable a dispositivos de escritorio y celulares, para esta primer iteración se determina la implementación de una aplicación web con diseño responsive mobile-first.

Decisiones arquitectónicas

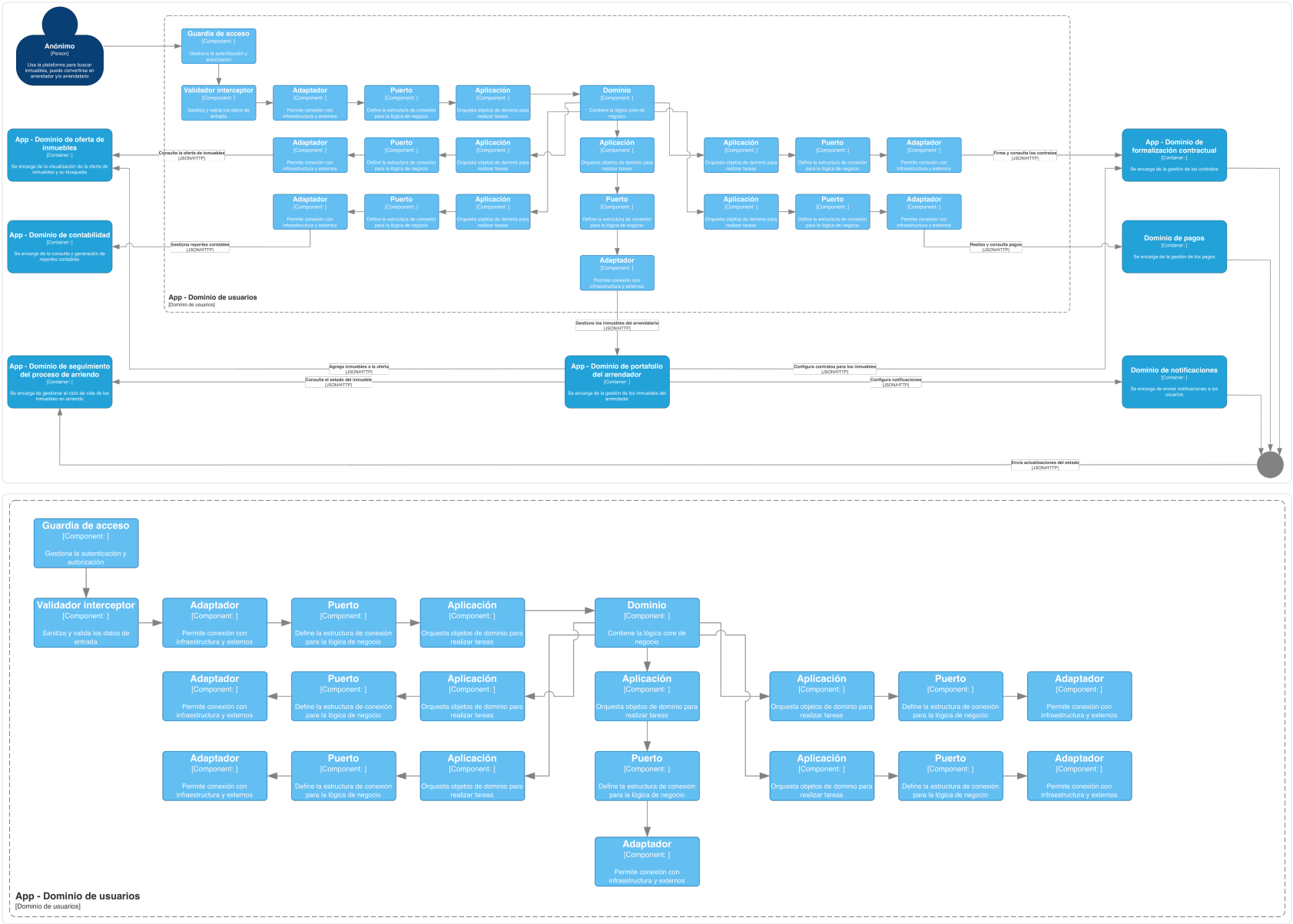
ID	Título	Contexto	Decisión	Consecuencias
AD-03	Tipo de aplicación	Según los hallazgos, el usuario espera utilizar principalmente una solución para dispositivos celular, pero también para dispositivos de escritorio.	Aplicación web responsive con diseño mobile-first	Requiere un esfuerzo adicional el hacer la aplicación responsiva adecuadamente a diferentes tamaños de pantalla.
AD-04	Arquitectura de los módulos de aplicación	Se espera balance entre una implementación no muy costosa pero al mismo tiempo mantenible y evolucionable.	Arquitectura hexagonal	Solución que desacopla la lógica de negocio de la infraestructura lo que permite en teoría evolucionar la aplicación a otra arquitectura mucho más fácil
AD-05	Comunicación	En pro de una posible evolución se quiere mantener un acoplamiento bajo y dominios bien delimitados.	Comunicación a través de APIs a pesar de estar en una misma unidad de despliegue.	Puede aumentar la latencia del sistema.
AD-06	Lenguaje de programación	La simplicidad y bajo costo debe verse no solo en el diseño de	Stack basado en Javascript para aprovechar su integración nativa	Se podría reducir el desempeño del backend en comparación al uso

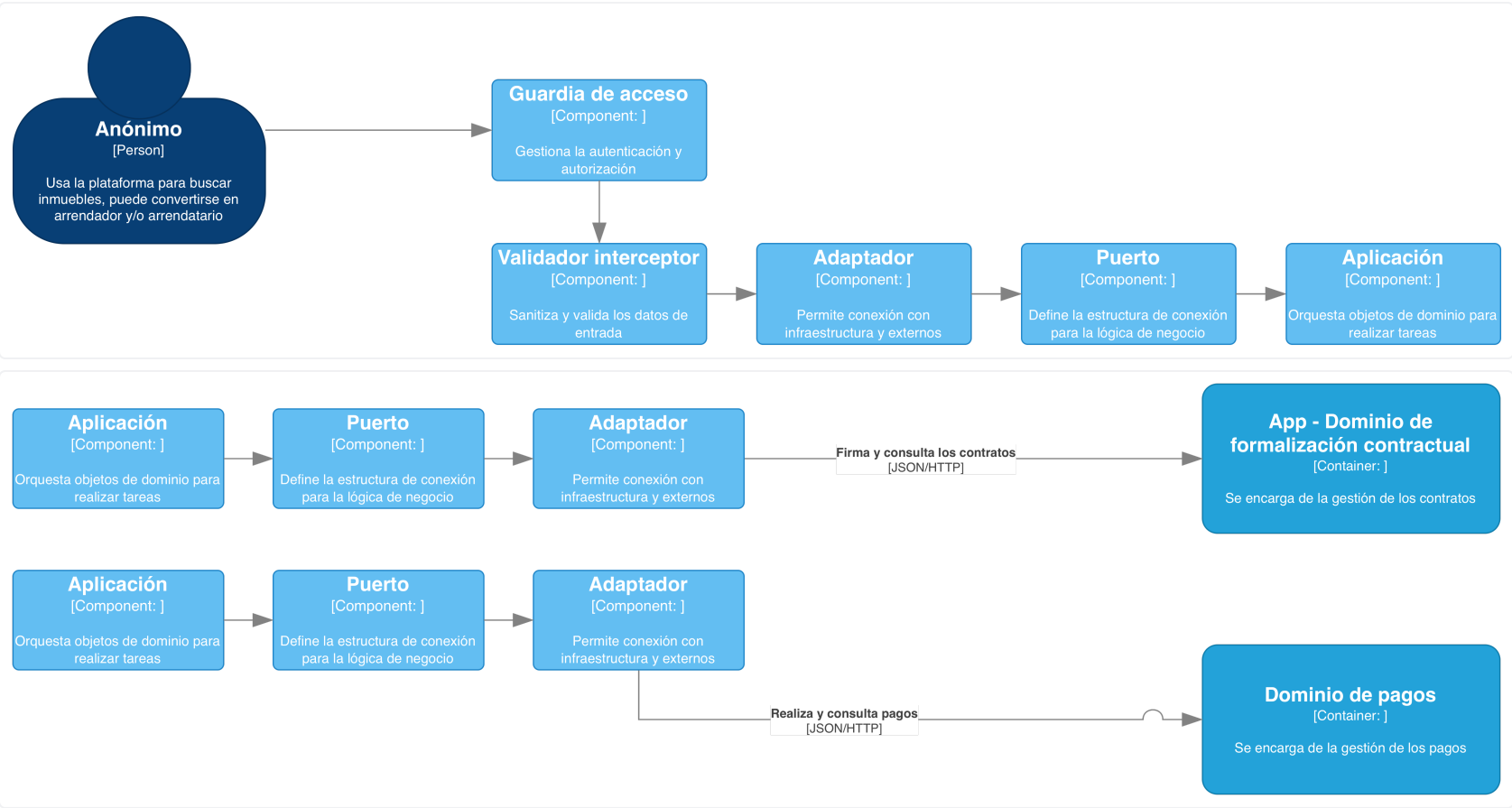
		arquitectura sino también en el diseño detallado.	en clientes web y se escoge para backend para no complejizar el desarrollo con mas de un lenguaje.	de otros lenguajes de programación.
AD-07	Disponibilidad en servicios que operan con terceros	Dado que el sistema requiere múltiples integraciones con sistemas externos, se espera que una degradación de uno de estos servicios no ocasione fallas en el comportamiento de la aplicación.	Circuit breaker en los adaptadores para integraciones con terceros con lógica de backoff exponencial y fallback en circuito abierto	Permite controlar la cantidad de llamados a los externos durante periodos de fallo reduciendo latencia y evitando indisponibilidad de la aplicación
AD-08	Validación de los datos de entrada	Para evitar ataques de scripting y prevenir errores de usuario se deben validar los datos que ingresan desde externos hacia la aplicación.	Sanitización y validación de los datos de entrada en UI y API	Aumenta la latencia y reduce el desempeño pero en la UI mejora la experiencia del usuario con la prevención de errores y mitiga ataques de seguridad.
AD-09	Control de acceso	Las funcionalidades a las que un usuario puede acceder en el sistema dependen de los roles que tiene.	Role Based Access Control (RBAC) + resource ownership	Permite mapear el comportamiento del negocio a la aplicación respecto a los roles pero se agrega una capa de validación respecto a que cada usuario solo puede ver recursos que le pertenezcan. Se debe tener cuidado en el

mantenimiento y gestión adecuada de estos permisos. Si se realiza adecuadamente podría evolucionarse a Attribute Based Access Control (ABAC) que reduce la superficie de ataque al tener controles más granulares.

Vistas

A continuación se presenta la vista explosionando o haciendo zoom del módulo de aplicación del dominio de usuarios haciendo uso de la arquitectura propuesta excluyendo únicamente una representación explícita de la UI y del circuit breaker.





Iteración 3

Objetivo

Definir la **arquitectura de datos y almacenamiento** para el monolito modular MVP

Drivers

- Seguridad
- Desempeño
- Disponibilidad

Elemento a descomponer

El **sistema a nivel de gestión de datos**

Conceptos de diseño

Para esta tercera iteración, al tratarse de definir una línea base de gestión de datos, se tomaran los siguientes conceptos de diseño:

Concepto de diseño	Descripción	Beneficios	Trade-offs
Estilos			
Bases de datos independientes por módulo	Cada modulo tiene su propia base de datos.	Alto desacoplamiento; Alta autonomía evolutiva.	Alta complejidad de implementación y de generación de reportes por la distribución de la data en diferentes bases de datos.
Base de datos única con	Una sola instancia pero cada módulo	Desacoplamiento medio; Complejidad	Si se afecta la instancia se degrada

esquemas independientes por módulo	tiene separación lógica por esquemas.	media.	el servicio de todos los módulos. Si no se tiene cuidado se pueden acoplar los módulos al hacer joins entre las tablas de los diferentes esquemas.
Base de datos única	Una sola instancia de base de datos y normalmente un solo esquema lógico (tablas compartidas). Los módulos comparten el mismo modelo de datos.	Alta simplicidad.	Bajo desacoplamiento y puede dificultar la evolución a otros estilos más robustos.
Patrones			
CQRS	Este patrón consiste en separar la infraestructura y responsabilidades de consulta y escritura sobre la base de datos para tener hacer uso de bases de datos optimizadas en consulta y mejorar el desempeño.	Mejora el desempeño; Permite escalar mejor dependiendo de qué operación requiere más recursos (escritura o lectura).	Se puede presentar inconsistencia eventual; Mayor complejidad con respecto a un modelo de una sola base de datos.
Data Lake	Repositorio central (generalmente object storage) que almacena datos en bruto o semi-procesados (logs, eventos, dumps) de múltiples fuentes. Se usa para exploración, ML, analítica flexible.	Flexibilidad al poder guardar cualquier tipo de dato crudo; Ideal para datos no estructurados.	Se debe considerar un esfuerzo adicional en la limpieza y transformación de los datos. Si no se tienen catálogos de datos se complejiza su administración y gobernanza.
Data Warehouse	Repositorio analítico estructurado, con datos integrados y	Única fuente de verdad para analítica; Mejor	Mayor complejidad al requerir modelar los datos e

	modelados (p. ej., esquema estrella: hechos/dimensiones) para reportería confiable (BI, dashboards, KPIs).	desempeño en consultas para BI; Mejor calidad en la información y gobernanza de los datos.	involucrar procesos de limpieza y transformación de los datos.
Tácticas			
Caché	Esta táctica consiste en involucrar un sistema alternativo de almacenamiento optimizado para consultas que permite traer datos que no cambian frecuentemente a altas velocidades.	Mejora el rendimiento al mejorar la velocidad de consulta; Reduce la carga del servidor al evitar tener que tratar completamente la solicitud.	Si se definen tiempos de vida muy altos o no se gestiona correctamente la actualización de datos cuando estos cambien se pueden convertir en datos obsoletos que afecten el correcto funcionamiento de la aplicación.
Familias tecnológicas			
Bases de datos Relacionales	Modelo tabular con esquema definido, relaciones, transacciones ACID, consultas SQL y constraints.	Consistencia fuerte; Integridad referencial; Madurez tecnológica en backups, replicación y observabilidad.	Complejidad en el cambio de modelos/esquemas de datos.
Bases de datos no relaciones	Familia de modelos: documento, key- value, columna, grafo.	Esquema flexible; Escalado horizontal optimizado.	Puede presentarse consistencia eventual. Al almacenar esquemas flexibles puede requerir un trabajo adicional el realizar join entre las estructuras de datos.

Considerando que en esta tercera iteración el foco es definir una arquitectura de datos coherente con un MVP evolutivo, se selecciona una **única instancia de base de datos** para el monolito modular, pero con **separación lógica por esquemas independientes por módulo**, lo que reduce la complejidad operativa inicial y, al mismo tiempo, deja una ruta clara para evolucionar hacia **bases de datos por módulo**. Para mejorar el desempeño de lecturas frecuentes, se incorpora **caché** para datos principalmente estáticos o de baja frecuencia de actualización, evitando recargar la base transaccional y estabilizando tiempos de respuesta. Adicionalmente,

dado que el sistema gestiona contenido como **documentos (TyC, contratos, comprobantes) e imágenes**, se adopta **almacenamiento de objetos** como repositorio primario para archivos. Finalmente, se define una estrategia híbrida para soportar flexibilidad y auditabilidad: se utiliza una base de datos **relacional**, pero registrando la información de entrada también en una capa **raw** en formato **JSON/JSONB**, desde la cual se ejecutan procesos programados (cron jobs) de tipo **ETL** que transforman la data cruda a un modelo **curado y tipado** (columnas por dato) utilizado por los módulos para operaciones de lectura; con esto se habilita trazabilidad completa y capacidad de reprocesamiento, ya que ante cambios de esquema se conserva el histórico raw y es posible eliminar y reconstruir las tablas curadas con el nuevo modelo sin pérdida de información.

Sub elementos

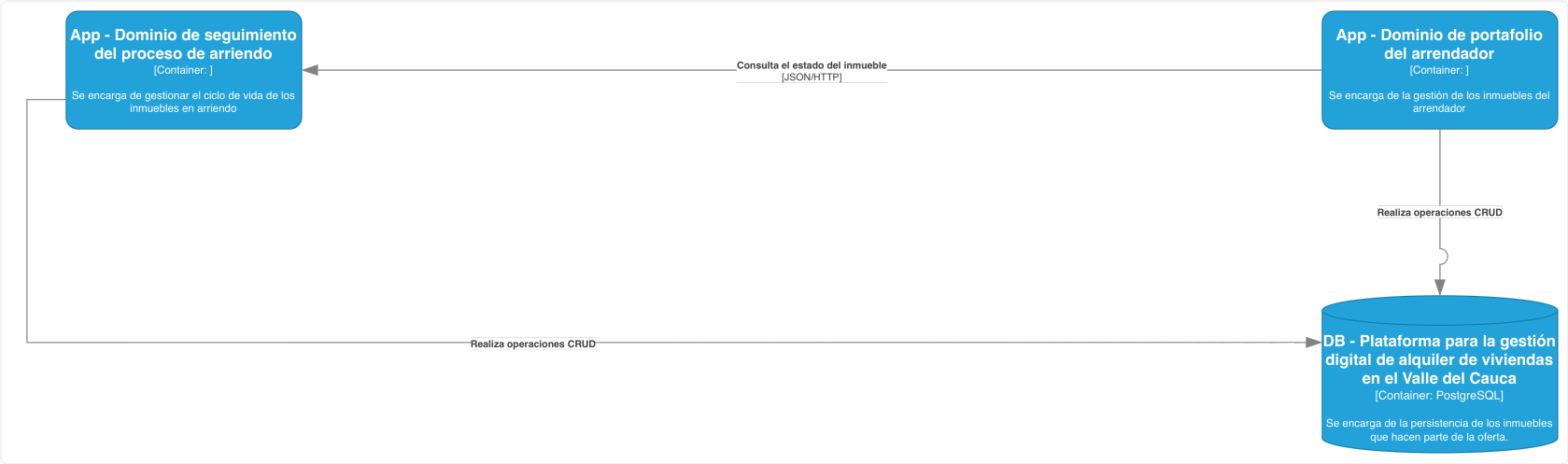
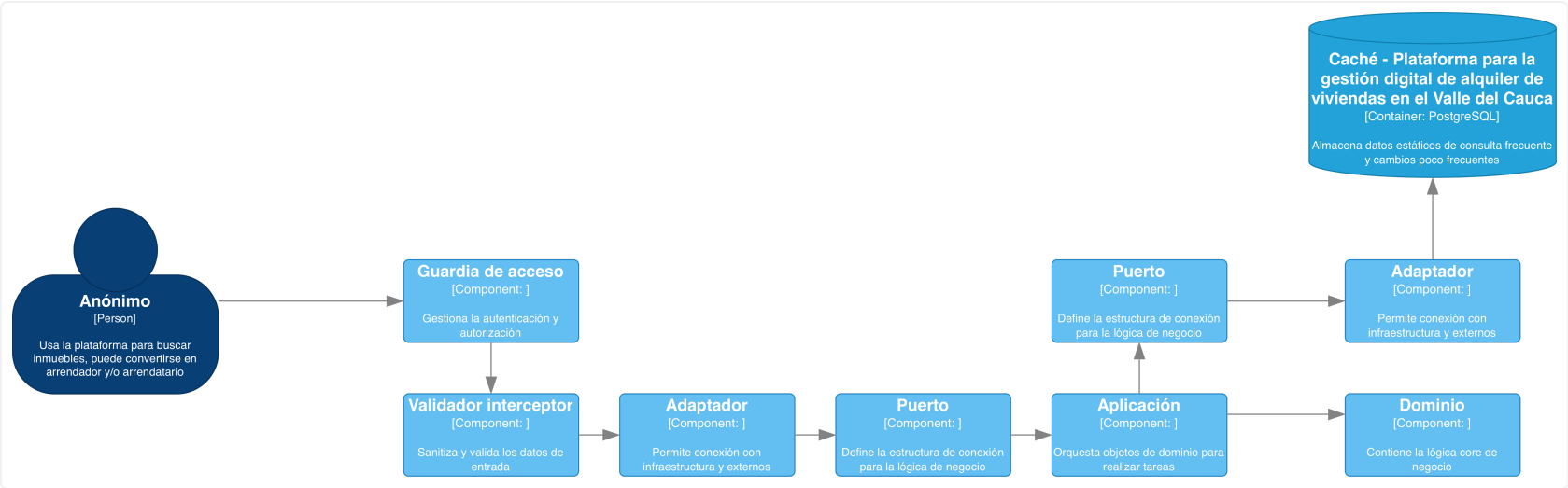
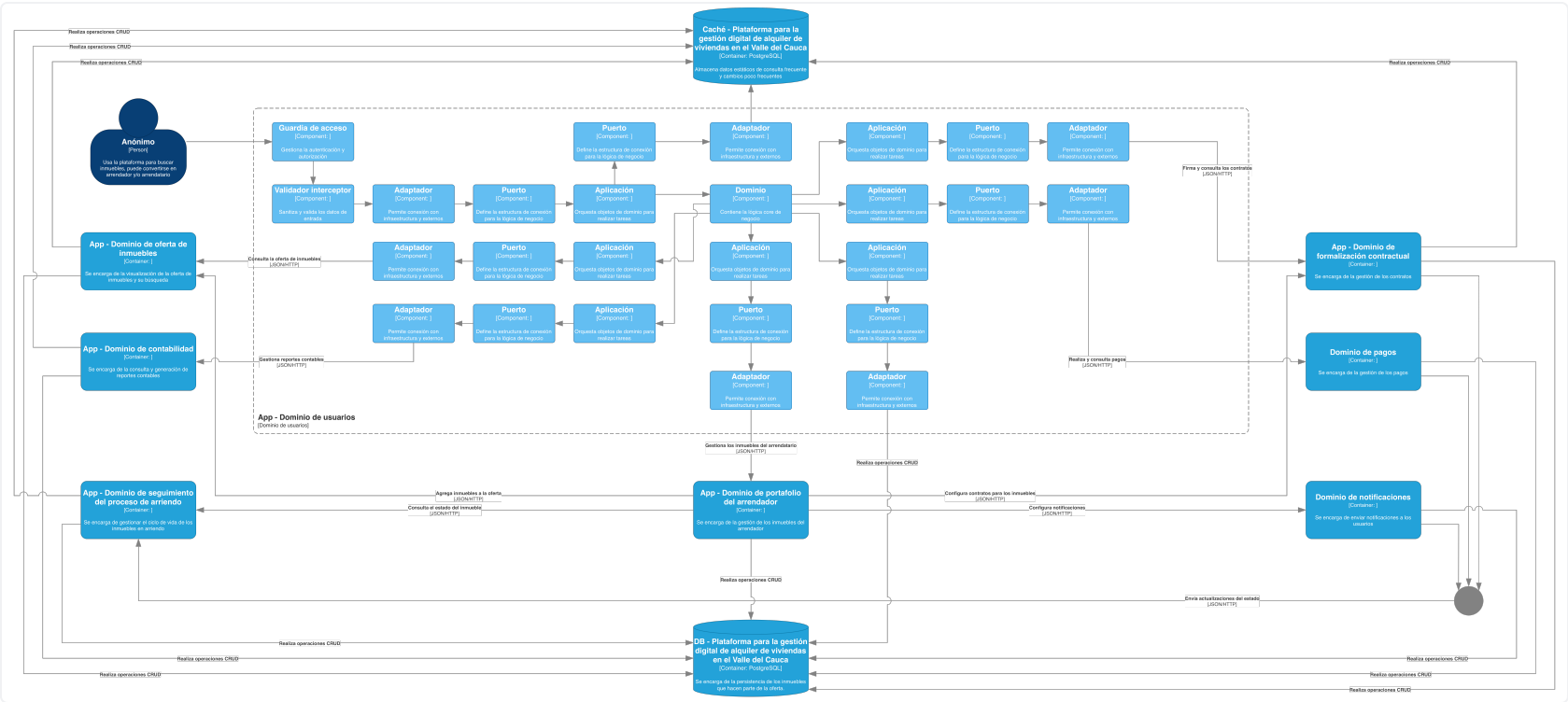
Dominios de línea base, es decir, todo el resto de dominios con casos más específicos también van a cumplir o heredar estos sub elementos de la línea base:

- Esquema de base de datos para separación lógica entre dominios.
- Caché de datos estáticos con baja frecuencia de actualización.
- Almacenamiento de objetos para contenido multimedia requerido por la aplicación
- Bases de datos relacionados con modelos específicos de escritura de datos "crudos" y modelos específicos de lectura para datos "limpios".

Decisiones arquitectónicas

ID	Título	Contexto	Decisión	Consecuencias
AD-10	Gestión de datos	En pro de una posible evolución se quiere mantener un acoplamiento bajo y dominios bien delimitados.	Bases de datos en esquemas separados por cada módulo.	Puede aumentar un poco la complejidad pero permite preparar el camino para evolucionar a base de datos por módulo.
AD-11	Desempeño en consultas que no cambian con alta frecuencia	Los tiempos de espera se dan por sentados como "cortos" por parte de los usuarios lo que implica que entre más rápido cargue la información mejor se puede percibir el desempeño y esto tiene impacto en la experiencia del usuario.	Caché en apis y en contenido que no cambie con frecuencia para reducir tiempos de espera o carga en el procesamiento del sistema.	Respuestas más rápidas en consultas estáticas, pero se debe controlar adecuadamente el TTL de los recursos cacheados.
AD-12	Implementación del caché	En escenarios de escalabilidad horizontal o de reemplazo de instancias por degradación/fallas (restarts, autoscaling, despliegues), un caché en memoria por	Implementar un caché distribuido usando Redis como capa compartida para almacenar respuestas/datos	Mejora la consistencia del caché entre módulos e instancias, reduce tiempos de respuesta. Como trade-off, introduce dependencia de infraestructura

		instancia produciría resultados inconsistentes y pérdida de caché al rotar instancias, afectando latencia percibida y estabilidad.	estáticos o semi-estáticos, aplicando un patrón cache-aside (leer del caché; si no existe, consultar fuente y poblar con TTL).	adicional y requiere definir TTL e invalidación, y manejo de fallos.
AD-13	Persistencia híbrida: capa RAW y capa curada para lecturas	Se requiere flexibilidad ante cambios de esquema, preservación histórica y trazabilidad; además, los módulos necesitan un modelo tipado/limpio para lecturas eficientes	En base relacional, registrar entradas en una tabla RAW con JSON/JSONB (eventos/datos crudos) y mantener tablas curadas tipadas (columnas) para operación de lectura.	Permite reprocesar y reconstruir modelos curados ante cambios de esquema sin perder histórico, pero introduce complejidad operativa (jobs, backfills, control de versiones del payload).
AD-14	Motor de base de datos	Para el MVP se prioriza reducir riesgo y tiempo de implementación usando una tecnología madura y conocida por el equipo (familiaridad), sin perder capacidad de evolución. Además, la estrategia de persistencia híbrida requiere soporte eficiente para almacenar datos crudos (RAW) en formato semi-estructurado y luego transformarlos a modelos curados, manteniendo opciones de indexación y consulta sobre el contenido JSON.	Adoptar PostgreSQL como base de datos relacional principal, aprovechando su soporte nativo de JSON/JSONB, índices y capacidades de consulta sobre campos JSON para implementar la capa RAW y facilitar ETL hacia tablas tipadas/curadas.	Reduce el riesgo técnico y acelera el desarrollo por la familiaridad y el amplio ecosistema/herramientas disponibles; habilita la persistencia híbrida (RAW JSON/JSONB + curado relacional) con buen desempeño. Como trade-off, exige gobernanza de esquema (versionado del payload, límites de tamaño de JSON, particionado/retención del RAW) y diseño cuidadoso de índices para evitar degradación por crecimiento de datos semi-estructurados.



De este modo, las tres iteraciones de ADD permiten definir progresivamente la arquitectura del sistema: primero el estilo estructural del sistema (monolito modular), luego la arquitectura interna de la aplicación (hexagonal y mecanismos de seguridad), y finalmente la arquitectura de persistencia y gestión de datos, asegurando que todas las decisiones estén alineadas con los drivers de calidad identificados.

Riesgos arquitectónicos identificados

Riesgo	Descripción	Mitigación
--------	-------------	------------

Crecimiento de datos RAW	El almacenamiento continuo de JSON/JSONB puede aumentar rápidamente el tamaño de la base de datos.	Políticas de particionado, retención y archivado de eventos históricos.
Dependencia del caché	Si Redis falla, algunas consultas pueden degradar su desempeño.	Estrategia cache-aside con fallback a base de datos.
Complejidad del ETL	La lógica de transformación puede crecer con el tiempo.	Versionado de payload y pruebas automatizadas de pipelines ETL.

Puntos de Sensibilidad

Los **puntos de sensibilidad** son decisiones arquitectónicas cuyo cambio puede afectar significativamente uno o más atributos de calidad.

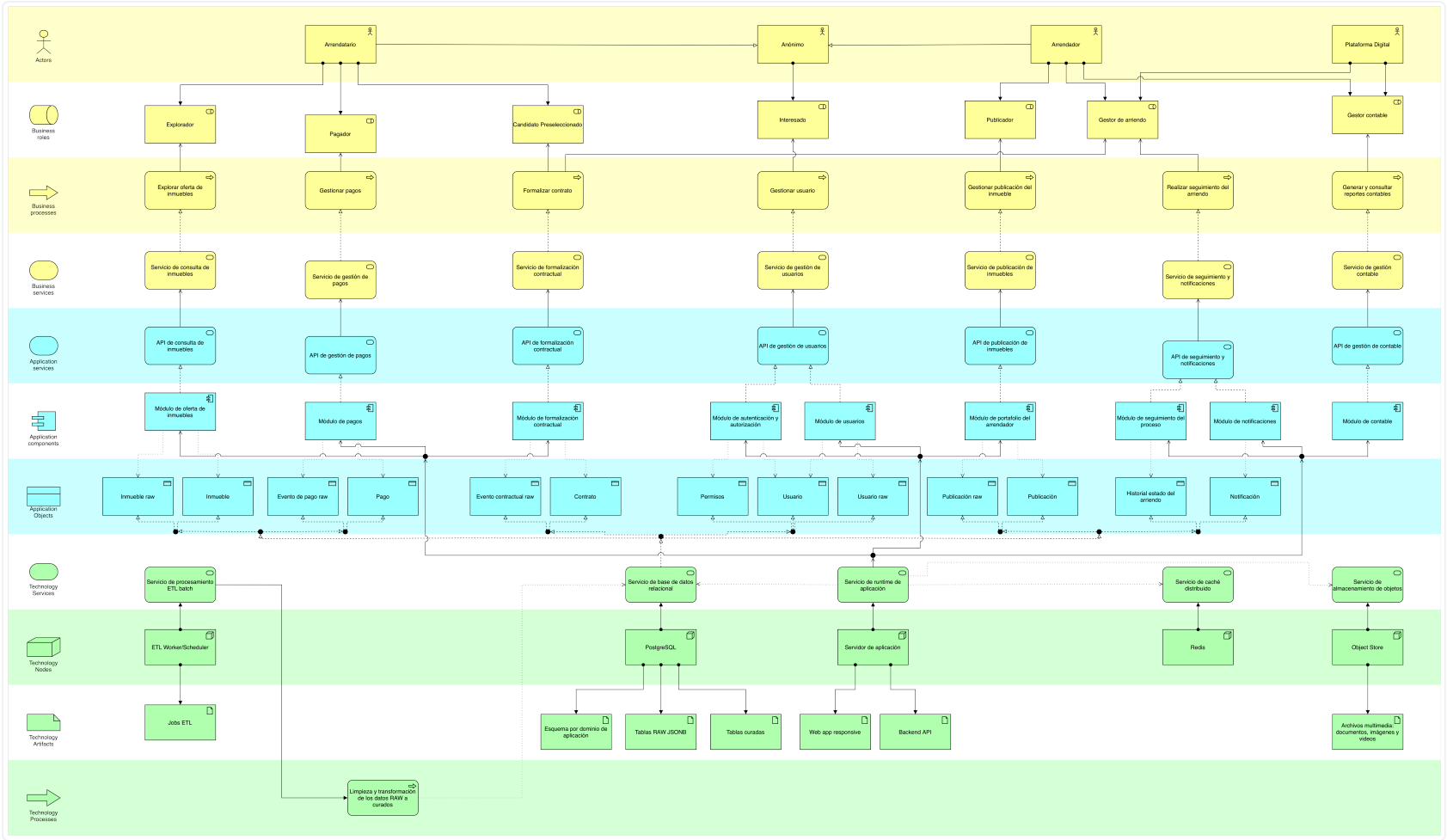
Decisión arquitectónica	Atributos afectados	Explicación
Uso de Redis como caché distribuido	Desempeño, escalabilidad	Si Redis no se utiliza o se configura incorrectamente, el sistema dependería exclusivamente de la base de datos para consultas frecuentes, aumentando latencias y carga en el sistema.
Persistencia híbrida (RAW JSON + modelo curado)	Evolución, mantenibilidad	Cambiar esta estrategia afectaría la capacidad de reprocesar datos históricos o adaptar el sistema a cambios de esquema.
Separación de esquemas por módulo	Evolución, mantenibilidad	Si todos los módulos compartieran el mismo esquema sin separación lógica, la evolución del sistema sería más compleja debido al mayor acoplamiento entre dominios.
Uso de almacenamiento de objetos para archivos	Escalabilidad, desempeño	Almacenar archivos en la base de datos incrementaría el tamaño de la base y afectaría operaciones de respaldo y rendimiento.

Trade-off Points

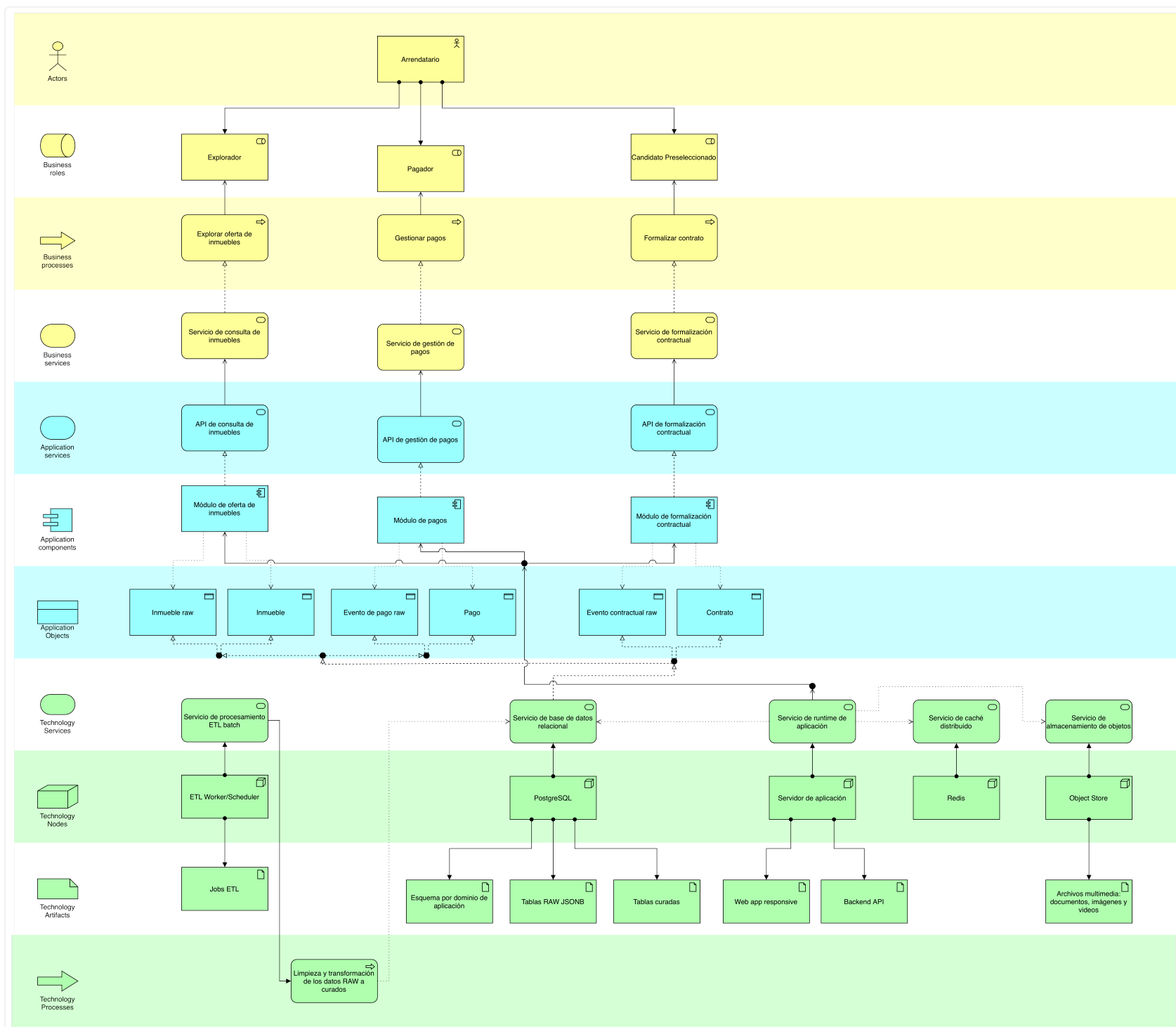
Los **trade-offs** representan decisiones que mejoran un atributo de calidad pero pueden impactar negativamente otro.

Decisión	Beneficio	Impacto
Caché distribuido con Redis	Reduce latencia y carga en base de datos	Introduce dependencia de infraestructura adicional
Persistencia híbrida (RAW + curado)	Permite evolución del esquema y trazabilidad histórica	Incrementa complejidad operativa mediante procesos ETL
Monolito modular	Reduce complejidad inicial y costo operativo	Puede requerir refactorización futura para microservicios
ETL programado en lugar de streaming	Reduce complejidad del MVP	Introduce latencia entre datos crudos y modelo curado

Diagrama final



Haciendo un zoom a un fragmento del diagrama para mejor visibilidad:



Conclusión

El análisis muestra que la arquitectura propuesta prioriza simplicidad operativa y evolución progresiva, lo cual es consistente con el objetivo de desarrollar un **MVP evolutivo**. Las decisiones tomadas equilibran atributos de calidad como desempeño, mantenibilidad y evolución, aceptando ciertos compromisos controlados, como la introducción de procesos ETL o dependencia de componentes adicionales como Redis. Sin embargo, estas decisiones mantienen abierta la posibilidad de evolucionar hacia arquitecturas más complejas si el crecimiento del sistema lo requiere.

Diseño del modelo de datos

Introducción

Propósito del documento

Esta sección describe el proceso de modelado de datos del sistema de gestión digital del ciclo de arriendo, estableciendo la estructura conceptual y lógica del dominio, así como las decisiones de diseño que permiten soportar los procesos operativos, contractuales, financieros y de seguimiento asociados. Su objetivo es proporcionar una base consistente para el desarrollo, mantenimiento y evolución del sistema, asegurando coherencia entre los requerimientos del negocio y la implementación técnica del modelo de datos.

Alcance del modelado de datos

El modelado abarca las entidades, relaciones, catálogos y estructuras necesarias para representar el ciclo completo de oferta, formalización, ejecución y seguimiento de contratos de arriendo, incluyendo la gestión de inmuebles, portafolios, contratos, estados operativos, pagos programados, notificaciones y reportería contable. Se consideran tanto estructuras operativas orientadas a transacciones como estructuras analíticas orientadas a consulta eficiente y trazabilidad histórica.

Contexto del dominio de datos

Visión general del dominio de negocio

El dominio corresponde a la gestión digital integral del ciclo de arriendo de inmuebles, abarcando desde la publicación de ofertas, la administración del portafolio del arrendador, la formalización contractual, el seguimiento del estado operativo del arriendo y la gestión de pagos asociados. El modelo de datos soporta tanto la operación transaccional diaria como la consolidación de información histórica y analítica necesaria para la toma de decisiones y la trazabilidad del proceso.

Objetivos funcionales que impactan el modelo de datos

El modelo debe permitir representar de forma estructurada los procesos de registro y administración de inmuebles, gestión de usuarios y roles, formalización y firma de contratos, programación y registro de pagos, seguimiento de estados del arriendo y notificación de eventos relevantes. Asimismo, debe soportar la generación de métricas financieras y operativas, así como la persistencia de historiales que reflejen la evolución del ciclo de vida del arriendo.

Objetivos no funcionales relevantes

Desde la perspectiva de datos, el diseño debe garantizar consistencia transaccional, integridad referencial y trazabilidad histórica de los cambios en estados contractuales, operativos y financieros. Adicionalmente, debe facilitar la eficiencia en consultas operativas y analíticas, permitir la evolución modular del sistema mediante separación por dominios y soportar integraciones futuras con servicios externos o componentes analíticos sin comprometer la estabilidad del modelo.

Modelo conceptual de datos

Objetivo del modelo conceptual

El modelo conceptual tiene como propósito representar de forma abstracta y comprensible las entidades principales del dominio del arriendo y sus relaciones fundamentales, sin considerar detalles técnicos de implementación. Este modelo permite alinear el entendimiento del negocio entre stakeholders técnicos y no técnicos, establecer los límites conceptuales del sistema y servir como base para la posterior definición del modelo lógico.

Identificación de entidades de negocio principales

Las entidades principales representan los conceptos fundamentales del dominio, incluyendo los inmuebles como activos gestionables, las unidades dentro del portafolio del arrendador, los ciclos de arriendo, los contratos que formalizan dichos ciclos, las obligaciones de pago y los eventos asociados a su ejecución. Asimismo, se consideran actores del sistema, mecanismos de notificación y estructuras de clasificación necesarias para la gestión del ciclo completo del arriendo.

Inmueble (Property)

Representa el activo físico susceptible de ser gestionado, publicado o arrendado dentro del sistema.

Define las características estructurales del bien inmobiliario, tales como su tipología, atributos físicos relevantes y ubicación.

Esta entidad constituye la base conceptual sobre la cual se construyen los procesos de gestión del portafolio y los ciclos de arriendo.

Unidad de portafolio (Portfolio Unit)

Representa la instancia gestionada de un inmueble dentro del portafolio de un arrendador.

Permite modelar la relación entre el propietario o gestor y el activo inmobiliario, separando la existencia del inmueble como entidad física de su administración operativa dentro del sistema.

Esta entidad facilita la gestión de inventario inmobiliario y la evolución histórica de los ciclos de arriendo asociados a un mismo activo.

Publicación comercial (Listing)

Representa la exposición comercial de una unidad de portafolio hacia potenciales arrendatarios.

Su existencia es opcional respecto a la unidad gestionada, permitiendo distinguir entre inmuebles administrados internamente y aquellos disponibles en el mercado.

Esta entidad soporta el proceso de descubrimiento de oferta y el inicio potencial del ciclo de arriendo.

Ciclo de arriendo (Lease)

Representa el período temporal durante el cual una unidad de portafolio es ocupada bajo condiciones contractuales definidas.

Constituye el agregado principal del dominio operativo del arriendo, integrando la información relativa al arrendatario, los estados del proceso, los contratos asociados y las obligaciones financieras derivadas.

Esta entidad permite modelar la sucesión de arriendos históricos sobre un mismo activo.

Contrato (Contract)

Representa el acuerdo legal que formaliza las condiciones del ciclo de arriendo.

Incluye la relación entre las partes involucradas, la documentación asociada, los estados de formalización y las evidencias generadas durante el proceso de firma.

Esta entidad garantiza la trazabilidad jurídica del arriendo y soporta la gestión documental del sistema.

Obligación de pago programada (Scheduled Payment)

Representa el compromiso financiero derivado del contrato de arriendo, establecido en términos de monto y fecha de vencimiento.

Permite estructurar el calendario financiero del ciclo de arriendo y establecer el marco de referencia para la ejecución posterior de pagos.

Pago (Payment)

Representa la ejecución efectiva de una obligación financiera programada.

Se vincula a eventos operativos del sistema, estados financieros y registros provenientes de integraciones con servicios externos.

Esta entidad permite reflejar el comportamiento financiero del ciclo de arriendo y soporta la generación de historial y métricas asociadas.

Estado del proceso (Status)

Representa las clasificaciones que describen la evolución del ciclo de vida de entidades operativas, contractuales o comerciales.

Incluye estados asociados a publicaciones, ciclos de arriendo, contratos, pagos y procesos de firma.

Estas estructuras permiten controlar transiciones de negocio y mantener coherencia semántica en el sistema.

Usuario (User)

Representa las personas que interactúan con el sistema en distintos roles, tales como arrendadores, arrendatarios o usuarios administrativos.

Esta entidad permite modelar la relación entre actores y artefactos del dominio, garantizando control de responsabilidades y trazabilidad de acciones.

Notificación (Notification)

Representa los mecanismos de comunicación del sistema con los usuarios respecto a eventos relevantes del ciclo de arriendo.

Permite gestionar preferencias, tipos de notificación y registros históricos de comunicación.

Reporte financiero (Accounting Report)

Representa las estructuras de consolidación de información financiera derivada de los pagos asociados a unidades de portafolio.

Estas entidades permiten soportar análisis de rendimiento económico y facilitar la toma de decisiones por parte de los arrendadores.

Relaciones conceptuales entre entidades

Las relaciones conceptuales reflejan la dependencia lógica entre los distintos componentes del dominio. Los inmuebles se vinculan al portafolio del arrendador como unidades gestionadas; dichas unidades pueden estar asociadas a publicaciones comerciales y a ciclos temporales de arriendo. Cada ciclo de arriendo puede generar contratos, obligaciones de pago y estados operativos, los cuales evolucionan a lo largo del tiempo y se documentan mediante historiales y evidencias asociadas.

Relación entre inmueble y unidad de portafolio

Un inmueble puede ser gestionado dentro de uno o varios contextos administrativos, representados mediante unidades de portafolio.

Esta relación permite separar la existencia física del activo de su gestión operativa, facilitando la trazabilidad histórica y la evolución del inventario inmobiliario.

Relación entre unidad de portafolio y publicación

Una unidad gestionada puede ser expuesta comercialmente mediante una publicación, aunque su existencia no depende de dicha exposición.

Esta relación permite distinguir entre inmuebles disponibles en el mercado y aquellos gestionados exclusivamente para control interno.

Relación entre unidad de portafolio y ciclo de arriendo

Una unidad de portafolio puede estar asociada a múltiples ciclos de arriendo a lo largo del tiempo, pero solo a uno activo en un momento determinado.

Esta relación introduce la dimensión temporal en el modelo y permite reconstruir la ocupación histórica del inmueble.

Relación entre ciclo de arriendo y contrato

Cada ciclo de arriendo puede estar formalizado mediante uno o varios contratos, dependiendo de la evolución del acuerdo entre las partes.

Esta relación soporta escenarios de renovación, modificación contractual o generación de evidencias adicionales.

Relación entre ciclo de arriendo y obligaciones financieras

Un ciclo de arriendo genera un conjunto de obligaciones de pago programadas que estructuran el compromiso financiero entre las partes.

Estas obligaciones representan el marco de referencia para la ejecución posterior de pagos.

Relación entre obligaciones programadas y pagos

Cada obligación financiera puede dar lugar a uno o varios eventos de pago que reflejan su cumplimiento.

Esta relación permite modelar el comportamiento financiero del arriendo y soportar la trazabilidad de ingresos.

Relación entre ciclo de arriendo y estados operativos

El ciclo de arriendo evoluciona a través de estados que representan su progreso desde la negociación inicial hasta su finalización.

Estos estados permiten controlar el flujo de negocio y mantener coherencia operativa.

Relación entre publicaciones y estados comerciales

Las publicaciones comerciales también poseen un ciclo de vida propio, independiente del ciclo de arriendo.

Esta relación permite modelar la disponibilidad del inmueble en el mercado y su interacción con la demanda.

Relación entre usuarios y artefactos del dominio

Los usuarios del sistema se vinculan a las distintas entidades según su rol en el proceso, ya sea como propietarios o arrendatarios.

Esta relación permite estructurar la gobernanza de datos y garantizar trazabilidad de las interacciones.

Identificación de agregados de dominio

El modelo conceptual reconoce agregados que encapsulan coherencia transaccional y reglas de negocio específicas. Entre ellos se destacan el agregado del ciclo de arriendo, que agrupa contratos, estados operativos y obligaciones financieras, así como el agregado del portafolio del arrendador, que organiza la gestión de activos inmobiliarios. Otros agregados incluyen la gestión de publicaciones comerciales y la administración de identidades y notificaciones dentro del sistema.

Agregado de ciclo de arriendo (Lease Aggregate)

Este agregado representa el núcleo operativo del sistema y agrupa todas las entidades relacionadas con la ejecución temporal de un arriendo. Incluye el ciclo de arriendo como entidad raíz, los contratos que formalizan el acuerdo entre las partes, las obligaciones financieras derivadas, los pagos asociados y los estados que describen la evolución del proceso.

Este agregado define la coherencia transaccional del arriendo y concentra las reglas de negocio relacionadas con la ocupación de la unidad, la vigencia contractual y el cumplimiento de obligaciones financieras.

Agregado de portafolio inmobiliario (Portfolio Aggregate)

Este agregado organiza la gestión de los activos inmobiliarios desde la perspectiva del arrendador. Tiene como entidad raíz la unidad de portafolio, que encapsula la relación entre el inmueble físico y su administración dentro del sistema.

Permite estructurar el inventario de propiedades, gestionar su disponibilidad, asociar ciclos de arriendo históricos y consolidar información relevante para análisis y reportería.

Agregado de exposición comercial (Listing Aggregate)

Este agregado representa la dimensión comercial del sistema y tiene como entidad raíz la publicación de la unidad inmobiliaria. Incluye los atributos que describen la oferta, las características adicionales del inmueble desde una perspectiva de mercado y los estados que reflejan su visibilidad o disponibilidad.

Su independencia respecto al agregado de ciclo de arriendo permite modelar escenarios donde un inmueble puede estar publicado sin estar arrendado, o estar arrendado sin requerir exposición pública.

Agregado contractual (Contract Aggregate)

Este agregado encapsula la gestión documental y jurídica asociada a los acuerdos de arriendo. Incluye la entidad de contrato como raíz, así como los archivos asociados, los participantes del proceso contractual y los registros de firma electrónica o eventos relevantes del proceso de formalización.

Este agregado permite garantizar trazabilidad legal, control de versiones documentales y evidencia verificable del proceso contractual.

Agregado financiero (Payment Aggregate)

Este agregado gestiona las obligaciones económicas derivadas del ciclo de arriendo y su ejecución efectiva.

Incluye las obligaciones de pago programadas como estructura de referencia financiera y los pagos ejecutados como eventos que reflejan su cumplimiento.

Permite modelar el comportamiento financiero del arriendo, soportar el seguimiento de estados de pago y facilitar la generación de métricas financieras.

Agregado de identidad y acceso (Identity Aggregate)

Este agregado agrupa las entidades relacionadas con la gestión de usuarios, roles y permisos dentro del sistema. Permite controlar el acceso a funcionalidades, vincular actores con artefactos del dominio y mantener trazabilidad sobre las acciones realizadas.

Agregado de comunicación (Notification Aggregate)

Este agregado representa los mecanismos de interacción del sistema con los usuarios mediante notificaciones. Incluye las preferencias de comunicación, los tipos de notificación y los registros históricos de mensajes enviados.

Facilita la coordinación de eventos operativos del sistema con la experiencia del usuario y soporta la gestión de comunicaciones relevantes a lo largo del ciclo de arriendo.

Agregado analítico (Accounting Aggregate)

Este agregado agrupa las estructuras orientadas a la consolidación y análisis de información financiera derivada de los pagos asociados a unidades de portafolio. Incluye reportes individuales y agregados que permiten evaluar el desempeño económico de los activos inmobiliarios.

Su independencia respecto a los agregados operativos permite optimizar consultas analíticas sin afectar la consistencia transaccional del sistema.

Modelo lógico de datos

El modelo lógico de datos describe la estructura detallada de las entidades, relaciones, claves y reglas de integridad que permiten implementar de manera consistente el modelo conceptual del sistema. Su objetivo es traducir los conceptos del dominio en estructuras relacionales organizadas, garantizando coherencia transaccional, trazabilidad histórica y capacidad de evolución futura.

Estrategia de descomposición por dominios o esquemas

El modelo lógico se organiza mediante una descomposición modular basada en límites de dominio, donde cada uno se representa mediante un esquema lógico independiente. Esta estrategia permite surgir a partir del **AD-10**. Así la segmentación por dominios resultando en:

Dominio de oferta de inmuebles - Esquema: property_listings

Este esquema concentra la información asociada a la oferta de inmuebles y su representación pública o comercial. Incluye las estructuras necesarias para describir inmuebles desde la perspectiva de publicación, tales como atributos principales del bien, ubicación, material visual y características adicionales relevantes para la consulta del usuario.

Su propósito es soportar el descubrimiento, consulta y presentación de inmuebles disponibles, separando la dimensión comercial del activo de su administración interna dentro del portafolio del arrendador.

Dominio de portafolio del arrendador - Esquema: landlord_portfolio

Este esquema agrupa las entidades que representan la gestión patrimonial y operativa de los inmuebles desde la perspectiva del arrendador. Incluye la estructura del portafolio, las unidades administradas y los ciclos de arriendo asociados a dichas unidades.

Su función es modelar el inventario gestionado por el propietario y servir como núcleo de relación entre el activo inmobiliario, su explotación económica y su evolución histórica en términos de ocupación.

Dominio de seguimiento del proceso de arriendo - Esquema: tracking_process

Este esquema contiene las estructuras orientadas al seguimiento del estado operativo y comercial de procesos clave del sistema. Incluye estados actuales e históricos tanto para publicaciones como para ciclos de arriendo, así como sus respectivos catálogos de estados.

Su propósito es desacoplar la lógica de seguimiento y trazabilidad de estados respecto de las entidades operativas principales, permitiendo gestionar historiales, snapshots y evolución temporal del proceso de forma explícita y consistente.

Dominio de pagos - Esquema: payments

Este esquema concentra las entidades relacionadas con la gestión operativa de pagos, incluyendo obligaciones programadas, pagos ejecutados, estados de pago y registros asociados a interacciones con servicios externos. Su responsabilidad es soportar el flujo financiero transaccional del sistema, garantizando trazabilidad entre compromisos de pago, eventos financieros efectivamente realizados y evidencia operativa derivada de la integración con pasarelas o proveedores externos.

Dominio de contabilidad - Esquema: accounting

Este esquema agrupa estructuras orientadas a reportería y consolidación analítica de información financiera. Incluye reportes individuales y agregados contruidos sobre datos transaccionales previamente procesados. Su objetivo es optimizar consultas analíticas y soportar la evaluación del desempeño económico de las unidades inmobiliarias y portafolios, sin cargar innecesariamente el modelo transaccional principal.

Dominio de usuarios - Esquema: users

Este esquema reúne las entidades vinculadas a identidad, caracterización de usuarios y control de acceso. Incluye usuarios base, detalles diferenciados por tipo de persona, roles, permisos y relaciones de asignación entre ellos.

Su propósito es representar a los actores del sistema y controlar su interacción con los procesos del dominio, proporcionando una base consistente para autenticación, autorización y trazabilidad de responsabilidades.

Dominio de Notificaciones - Esquema: notifications

Este esquema concentra las estructuras necesarias para gestionar la configuración de comunicación del sistema con los usuarios. Incluye tipos de notificación y preferencias de notificación por usuario y canal.

Su función es soportar la personalización de comunicaciones asociadas a eventos del sistema, manteniendo desacoplada la configuración de notificaciones respecto de la lógica transaccional principal.

Dominio de formalización contractual - Esquema: contracts

Este esquema agrupa las entidades relacionadas con la formalización contractual del arriendo. Incluye contratos, partes contractuales, archivos asociados, tipologías documentales, estados documentales y registros del proceso de firma.

Su responsabilidad es modelar el ciclo de vida jurídico-documental del acuerdo de arriendo, garantizando integridad sobre evidencias, participantes, estados y artefactos asociados a la formalización legal.

Definición de entidades lógicas y atributos principales

Las entidades lógicas representan las estructuras persistentes que soportan los procesos del negocio.

Adicionalmente, el modelo incorpora catálogos normalizados para representar clasificaciones estandarizadas del dominio, como estados de procesos, tipos de documentos, tipos de notificación o categorías de atributos inmobiliarios. Estas entidades de referencia permiten mantener consistencia semántica, facilitar validaciones y simplificar la evolución de reglas de negocio sin necesidad de modificar estructuras operativas.

Entidades del esquema *property_listings*

properties

Representa la estructura base del inmueble desde la perspectiva de publicación y descripción. Contiene los atributos físicos principales del bien, tales como tipo de propiedad, dimensiones relevantes y características estructurales necesarias para su consulta y clasificación.

addresses

Representa la localización física asociada al inmueble. Su propósito es desacoplar los datos geográficos y de dirección respecto de la descripción principal de la propiedad, permitiendo mantener una estructura más organizada y flexible para búsquedas por ubicación.

listings

Representa la publicación comercial de una unidad inmobiliaria. Esta entidad concentra los datos visibles hacia el mercado, como título, descripción, precio, moneda y vigencia de la publicación, y se vincula a una unidad de portafolio para expresar que la oferta pública deriva de un activo gestionado por el arrendador.

photos

Representa los recursos visuales asociados a una publicación. Permite modelar múltiples imágenes por listing, conservar su orden de presentación y distinguir una imagen principal cuando sea necesario para la experiencia de consulta.

additional_features

Representa el catálogo de características adicionales que pueden enriquecer la descripción del inmueble. Su objetivo es soportar extensibilidad controlada en la caracterización de propiedades sin alterar continuamente la estructura base del modelo.

property_additional_features

Representa la asignación concreta de características adicionales a una propiedad específica. Permite registrar el valor asociado a cada característica configurable y mantener la relación entre el catálogo de features y el inmueble descrito.

Entidades del esquema *landlord_portfolio*

landlord_portfolio

Representa el portafolio de inmuebles administrado por un arrendador dentro del sistema. Actúa como contenedor lógico de las unidades inmobiliarias bajo gestión y permite agrupar los activos desde la perspectiva patrimonial y operativa del propietario o administrador.

portfolio_unit

Representa la unidad gestionada dentro del portafolio del arrendador. Vincula una propiedad concreta con su contexto de administración, separando el activo físico de su explotación operativa. Además, concentra información relevante para la gestión interna, como observaciones y canon base de referencia.

lease

Representa el ciclo temporal de arriendo asociado a una unidad de portafolio. Esta entidad materializa el período de ocupación de una unidad, vinculando arrendatario, fechas de vigencia y estado actual del arriendo, y funciona como eje de relación para contratos, pagos y seguimiento operativo.

Entidades del esquema *tracking_process*

lease_status

Representa el catálogo de estados posibles del ciclo de arriendo. Su propósito es normalizar las clasificaciones que describen la evolución operativa de un lease y garantizar consistencia semántica en el sistema.

lease_status_history

Representa el historial de cambios de estado de un lease. Registra las transiciones ocurridas a lo largo del tiempo y permite reconstruir la trayectoria operativa completa del ciclo de arriendo.

lease_current_status

Representa el estado vigente del lease en forma de snapshot operativo. Su función es optimizar la consulta del estado actual sin requerir recorridos constantes sobre el historial completo.

listing_status

Representa el catálogo de estados posibles de una publicación comercial. Permite normalizar el ciclo de vida del listing y controlar su disponibilidad o visibilidad dentro del sistema.

listing_status_history

Representa el historial de cambios de estado de una publicación. Su finalidad es conservar trazabilidad temporal sobre la evolución comercial del listing.

listing_current_status

Representa el estado vigente de la publicación en forma de snapshot. Permite acceder rápidamente a la condición actual del listing sin depender de la reconstrucción desde el historial.

Entidades del esquema *payments*

scheduled_payments

Representa las obligaciones de pago programadas para un ciclo de arriendo. Estructura el calendario financiero esperado, definiendo monto, moneda y fecha de vencimiento de cada compromiso económico derivado del lease.

payments

Representa los pagos efectivamente realizados en cumplimiento de una obligación programada. Vincula el evento financiero real con la obligación correspondiente y registra atributos clave como monto, fecha efectiva y estado del pago.

payment_status

Representa el catálogo de estados posibles para los pagos. Permite estandarizar las clasificaciones de los eventos financieros y mantener consistencia en reglas operativas, reportería y consultas de seguimiento.

payment_logs

Representa el registro de eventos asociados a la ejecución o procesamiento de pagos, especialmente en contextos de integración con proveedores externos. Su propósito es conservar evidencia operativa, referencias externas, plataforma de origen y datos relevantes para auditoría técnica o reprocesamiento.

Entidades del esquema *accounting*

aggregated_payment_reports

Representa estructuras de consolidación financiera a nivel agregado, orientadas a resumir el comportamiento económico de un portafolio dentro de una ventana temporal determinada. Su propósito es soportar reportería gerencial y análisis de desempeño financiero consolidado.

individual_payment_reports

Representa estructuras de consolidación financiera a nivel individual por unidad de portafolio. Permite analizar el comportamiento económico de cada activo específico, facilitando la evaluación de rendimiento y trazabilidad analítica a nivel de inmueble gestionado.

Entidades del esquema *users*

users

Representa la entidad base de identidad dentro del sistema. Contiene la información mínima necesaria para identificar al actor, autenticarlo y vincularlo con procesos del dominio, independientemente de si se trata de una persona natural o jurídica.

natural_person_details

Representa la caracterización extendida de usuarios que corresponden a personas naturales. Su objetivo es desacoplar la identidad base de los atributos específicos de este tipo de usuario.

legal_person_details

Representa la caracterización extendida de usuarios que corresponden a personas jurídicas. Permite registrar la razón social u otra información organizacional sin sobrecargar la entidad base de usuario.

roles

Representa el catálogo de roles funcionales del sistema. Permite agrupar permisos y estructurar el control de acceso desde una perspectiva de negocio.

permissions

Representa el catálogo de permisos específicos que gobiernan acciones sobre recursos del sistema. Su propósito es definir granularmente las capacidades autorizadas dentro de la plataforma.

users_roles

Representa la asignación de roles a usuarios. Permite expresar la relación entre actores y perfiles de acceso dentro del sistema.

roles_permissions

Representa la asignación de permisos a roles. Su función es materializar el modelo de autorización y desacoplar la definición de permisos de su aplicación a usuarios concretos.

Entidades del esquema *notifications*

notification_types

Representa el catálogo de tipos de notificación que el sistema puede emitir o configurar. Su propósito es normalizar las categorías de comunicación relevantes para el dominio.

notification_preference

Representa la configuración de preferencias de notificación por usuario. Permite registrar qué tipo de evento desea recibir un actor, a través de qué canal y si dicha preferencia se encuentra activa.

Entidades del esquema *contracts*

contracts

Representa el acuerdo formal que regula un ciclo de arriendo. Contiene la referencia al lease correspondiente, el período de vigencia contractual y el estado del contrato dentro de su ciclo de formalización.

contract_status

Representa el catálogo de estados posibles del contrato. Permite normalizar la evolución del documento contractual y controlar su estado dentro del proceso jurídico-documental.

contract_party

Representa las partes vinculadas a un contrato y el rol que cada una cumple dentro del acuerdo. Su propósito es modelar la participación de arrendador, arrendatario u otros actores relevantes en la formalización contractual.

files

Representa los archivos asociados al contrato, tales como borradores, documentos finales u otros artefactos documentales relevantes. Permite desacoplar la gestión documental del contrato base y soportar múltiples archivos por acuerdo.

file_types

Representa el catálogo de tipos de archivo contractuales. Su objetivo es clasificar documentalmente los artefactos asociados al contrato.

files_status

Representa el catálogo de estados posibles de los archivos contractuales. Permite controlar la vigencia o condición documental de cada artefacto almacenado.

signings

Representa los eventos o instancias de firma asociados a un contrato y a una parte contractual específica. Su finalidad es registrar el avance del proceso de firma, su marca temporal y la evidencia verificable del documento firmado.

signing_status

Representa el catálogo de estados del proceso de firma. Permite mantener consistencia semántica sobre el estado de cada evento de firma electrónica o proceso equivalente.

signing_logs

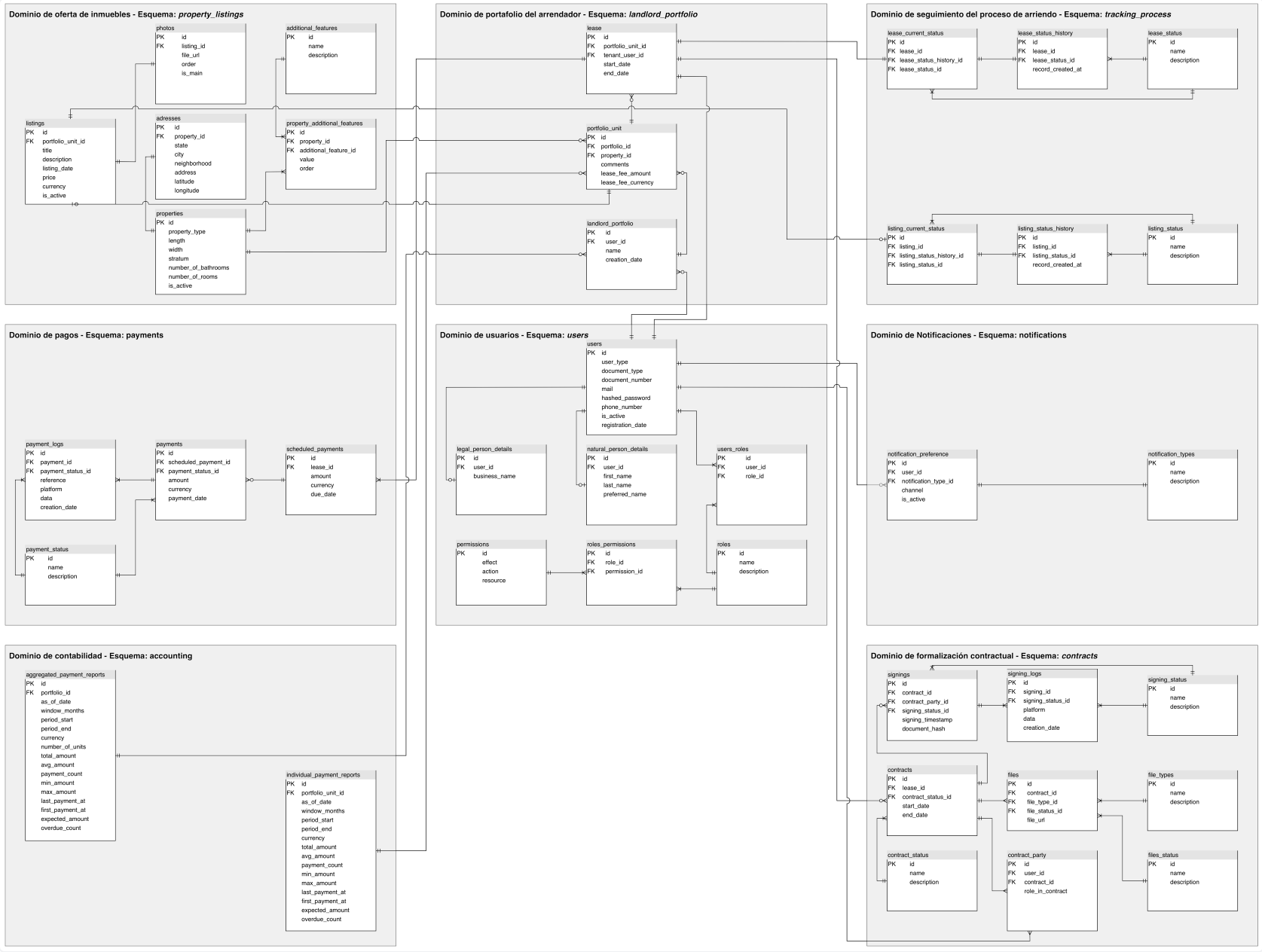
Representa el registro de eventos asociados al proceso de firma, especialmente aquellos derivados de interacciones con servicios externos. Conserva información operativa útil para auditoría, soporte técnico y trazabilidad del proceso documental.

Modelo físico

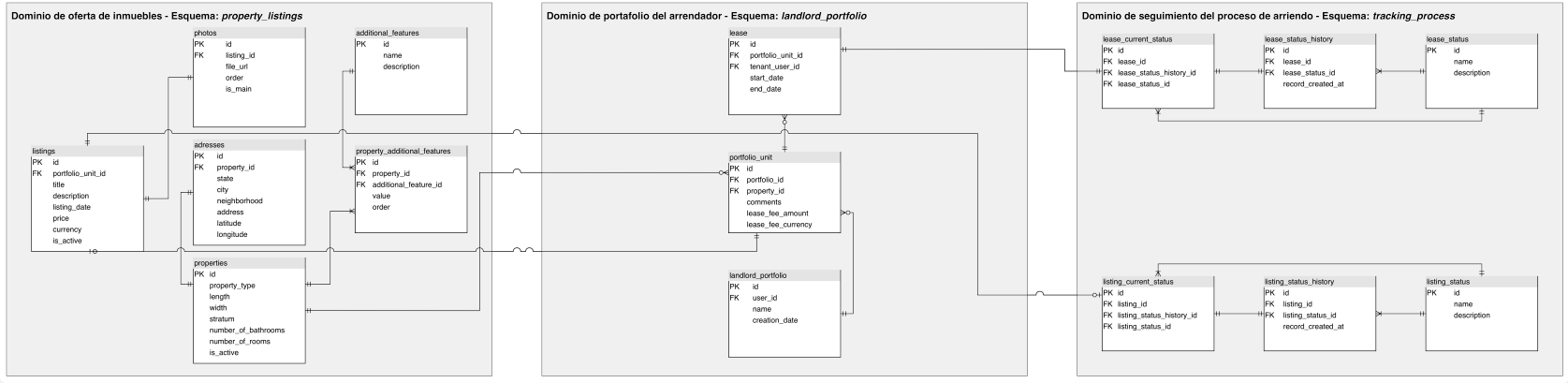
El modelo físico describe cómo se implementará el modelo lógico de manera concreta dentro del sistema de gestión de bases de datos. Esto se logra al materializar las entidades en forma de tablas, junto con claves primarias y foráneas que establecen las relaciones entre ellas.

Diagrama Entidad Relación (ERD)

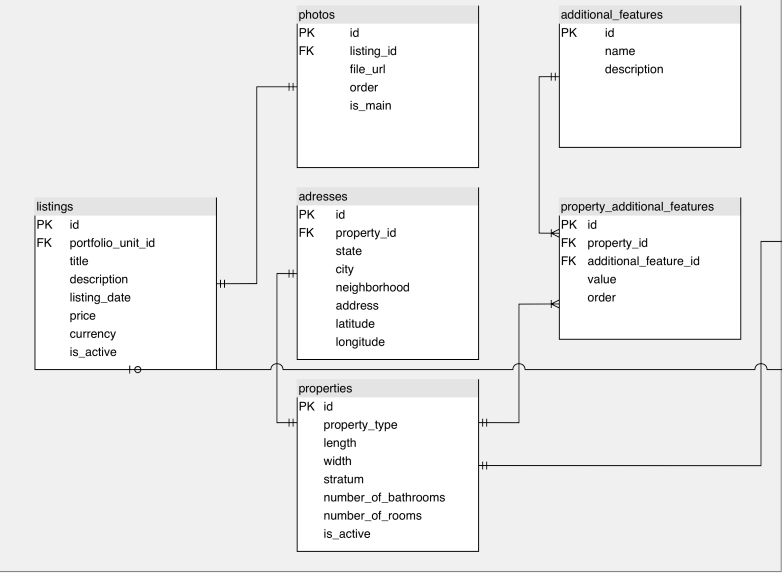
En este caso el modelo de datos se representa mediante un diagrama entidad-relación (ERD), el cual constituye la referencia principal para la estructura de datos implementada. Este diagrama refleja exclusivamente la primera versión de las tablas *curadas* que soportan los procesos operativos, analíticos y de control del sistema. No se incluyen las tablas de tipo *raw*, dado que estas corresponden a estructuras de ingestión técnica cuya composición se limita a un identificador único, un campo de datos semiestructurados y una marca temporal de creación. En consecuencia, el ERD presentado se enfoca en las entidades consolidadas del dominio, diseñadas para representar de forma explícita las reglas de negocio, las relaciones funcionales y la evolución operativa del sistema.



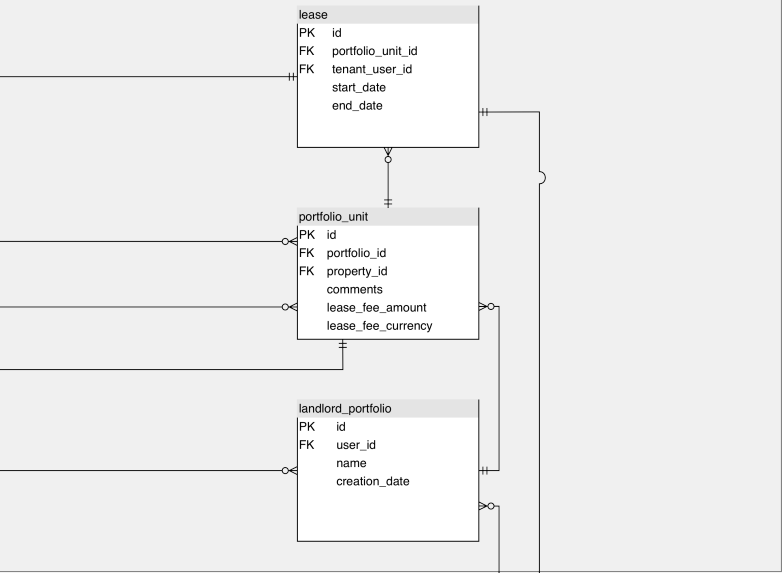
Haciendo zoom en algunos sectores del diagrama para mejor visibilidad:



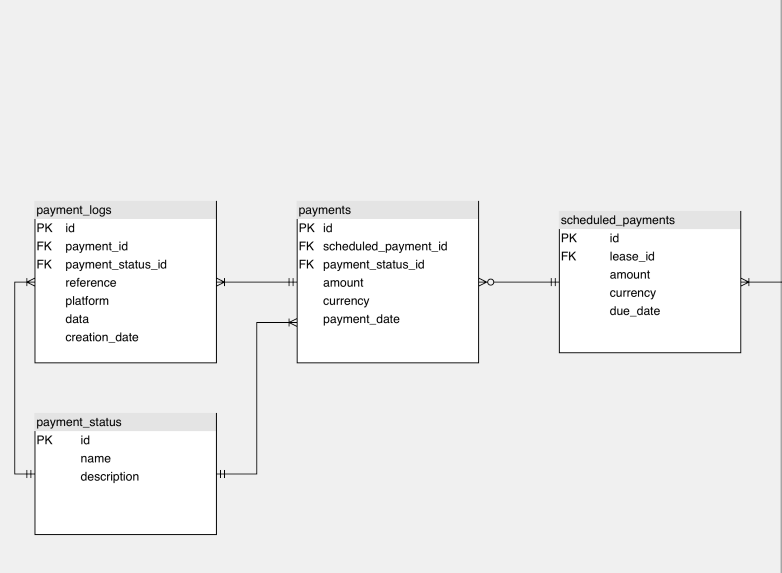
Dominio de oferta de inmuebles - Esquema: *property_listings*



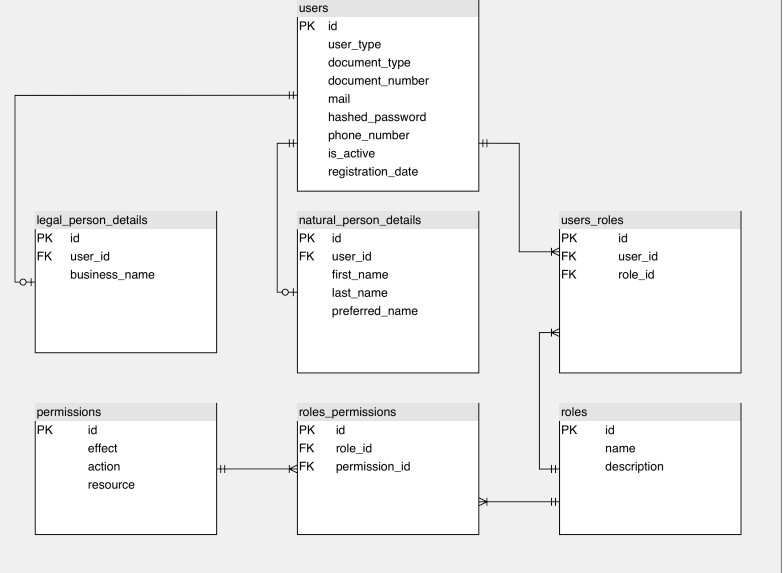
Dominio de portafolio del arrendador - Esquema: *landlord_portfolio*



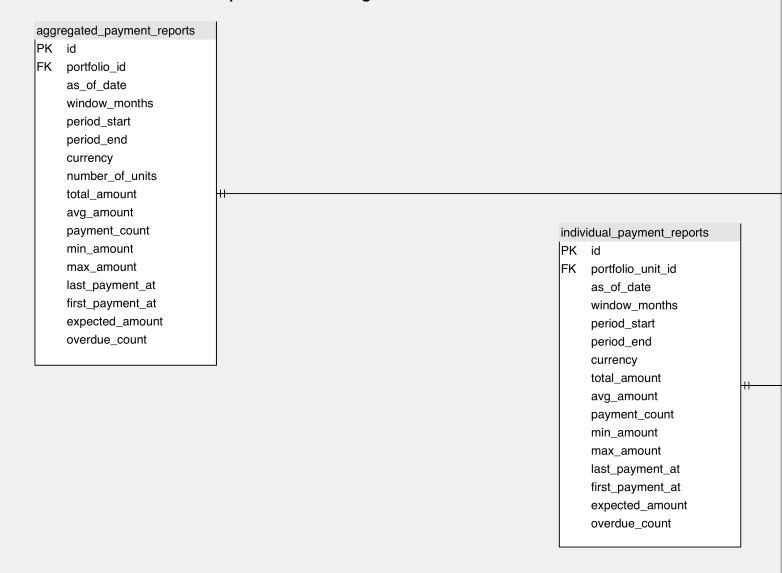
Dominio de pagos - Esquema: *payments*

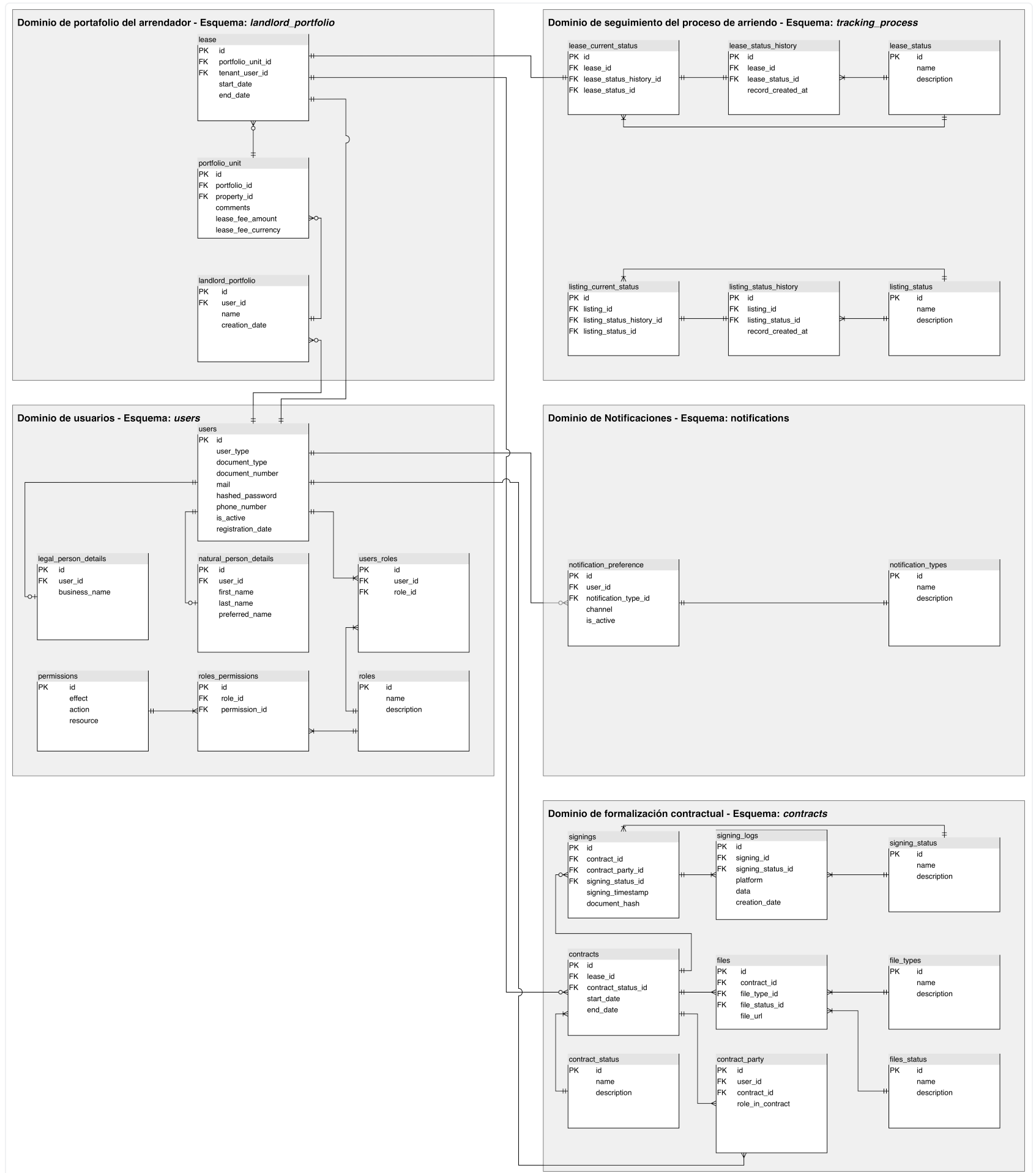


Dominio de usuarios - Esquema: *users*



Dominio de contabilidad - Esquema: *accounting*





Identificación de claves primarias y foráneas

Cada entidad lógica se define mediante una clave primaria que garantiza la unicidad de sus registros y permite su identificación dentro del sistema. Las relaciones entre entidades se establecen mediante claves foráneas que reflejan dependencias de negocio y aseguran integridad referencial.

Claves en el esquema *property_listings*

En este esquema, la entidad **properties** actúa como referencia estructural del inmueble y constituye una de las bases relacionales del modelo. La entidad **addresses** utiliza una clave foránea hacia **properties** para representar la localización física del inmueble sin mezclarla con la descripción base. La entidad **property_additional_features** utiliza claves foráneas hacia **properties** y **additional_features** para modelar una relación de asignación entre inmuebles y características configurables.

La entidad **listings** utiliza una clave foránea hacia **portfolio_unit**, estableciendo que toda publicación comercial debe originarse en una unidad administrada dentro del portafolio del arrendador. La entidad **photos** utiliza una clave foránea hacia **listings** para expresar la dependencia de los recursos visuales respecto de la publicación comercial.

Claves en el esquema *landlord_portfolio*

La entidad **landlord_portfolio** utiliza una clave foránea hacia **users**, expresando la relación entre el portafolio y el usuario que lo administra o posee. La entidad **portfolio_unit** utiliza claves foráneas hacia **landlord_portfolio** y **properties**, materializando el vínculo entre un portafolio específico y un inmueble estructuralmente descrito.

La entidad **lease** utiliza una clave foránea hacia **portfolio_unit**, indicando que todo ciclo de arriendo pertenece a una unidad administrada del portafolio. Adicionalmente, incorpora una clave foránea hacia **users** para identificar al arrendatario asociado al ciclo de arriendo.

Claves en el esquema *tracking_process*

Las entidades de catálogo **lease_status** y **listing_status** se identifican mediante claves primarias propias y sirven como referencia para las entidades operativas de estado. La entidad **lease_status_history** utiliza claves foráneas hacia **lease** y **lease_status**, permitiendo registrar la evolución histórica del estado de un ciclo de arriendo.

La entidad **lease_current_status** utiliza claves foráneas hacia **lease**, **lease_status_history** y **lease_status**, con el propósito de mantener un snapshot operativo consistente con el historial y con el catálogo de estados vigente. De manera análoga, **listing_status_history** utiliza claves foráneas hacia **listings** y **listing_status**, mientras que **listing_current_status** utiliza claves foráneas hacia **listings**, **listing_status_history** y **listing_status**.

Claves en el esquema *payments*

La entidad **scheduled_payments** utiliza una clave foránea hacia **lease**, estableciendo que cada obligación de pago pertenece a un ciclo de arriendo específico. La entidad **payments** utiliza claves foráneas hacia **scheduled_payments** y **payment_status**, vinculando cada pago realizado tanto con la obligación programada que pretende cubrir como con su estado financiero actual.

La entidad **payment_logs** utiliza claves foráneas hacia **payments** y **payment_status**, permitiendo registrar eventos operativos derivados del procesamiento del pago y asociarlos al estado correspondiente. La entidad **payment_status** actúa como catálogo de referencia para ambos casos.

Claves en el esquema *accounting*

La entidad **aggregated_payment_reports** utiliza una clave foránea hacia **landlord_portfolio**, permitiendo consolidar métricas financieras a nivel de portafolio. La entidad **individual_payment_reports** utiliza una clave foránea hacia **portfolio_unit**, con el objetivo de representar indicadores financieros a nivel individual por unidad gestionada.

Estas relaciones permiten mantener trazabilidad analítica respecto del portafolio y de cada activo específico, sin depender directamente de las tablas transaccionales en tiempo de consulta.

Claves en el esquema *users*

La entidad **users** funciona como raíz de identidad y es referenciada desde múltiples esquemas. Las entidades **natural_person_details** y **legal_person_details** utilizan claves foráneas hacia **users** para extender la

caracterización del actor según su tipo.

La entidad **users_roles** utiliza claves foráneas hacia **users** y **roles**, materializando la asignación de roles a usuarios. La entidad **roles_permissions** utiliza claves foráneas hacia **roles** y **permissions**, permitiendo estructurar el modelo de autorización a partir de permisos reutilizables agrupados en perfiles funcionales.

Claves en el esquema *notifications*

La entidad **notification_preference** utiliza claves foráneas hacia **users** y **notification_types**, representando la configuración de preferencias de notificación por actor y por tipo de evento. La entidad **notification_types** se comporta como catálogo de referencia y se identifica mediante su propia clave primaria.

Claves en el esquema *contracts*

La entidad **contracts** utiliza claves foráneas hacia **lease** y **contract_status**, indicando tanto el ciclo de arriendo al que pertenece el acuerdo como su estado actual de formalización. La entidad **contract_party** utiliza claves foráneas hacia **contracts** y **users**, permitiendo asociar partes contractuales concretas al acuerdo.

La entidad **files** utiliza claves foráneas hacia **contracts**, **file_types** y **files_status**, representando la clasificación y estado documental de cada archivo asociado al contrato. La entidad **signings** utiliza claves foráneas hacia **contract_party** y **signing_status**, permitiendo modelar la firma como un evento asociado a una parte contractual específica y gobernado por un catálogo de estados.

La entidad **signing_logs** utiliza claves foráneas hacia **signings** y **signing_status**, con el fin de registrar eventos operativos del proceso de firma y conservar evidencia temporal y técnica asociada.

Relaciones inter-esquema y trazabilidad del proceso

Las relaciones entre esquemas expresan el recorrido completo del dominio. La publicación comercial se origina en una unidad del portafolio; la unidad del portafolio puede dar lugar a uno o varios ciclos de arriendo; cada ciclo de arriendo puede originar contratos, obligaciones financieras y estados operativos; y estos a su vez alimentan procesos de reportería y comunicación.

Cardinalidades y restricciones de integridad

El modelo lógico especifica las cardinalidades entre entidades para representar correctamente las relaciones del dominio, tales como asociaciones uno a uno, uno a muchos o muchos a muchos. Estas cardinalidades permiten reflejar la multiplicidad real de los procesos del negocio, como la existencia de múltiples ciclos de arriendo sobre una misma unidad o múltiples obligaciones financieras dentro de un contrato.

Adicionalmente, se definen restricciones de integridad que garantizan la validez de los datos, incluyendo restricciones de unicidad, obligatoriedad de atributos clave, coherencia temporal y consistencia entre estados operativos.

Cardinalidades en el esquema *property_listings*

Una propiedad puede tener una única dirección asociada dentro del modelo actual, mientras que una dirección pertenece a una única propiedad. Una propiedad puede tener múltiples características adicionales asignadas y cada tipo de característica adicional puede aplicarse a múltiples propiedades, lo que da lugar a una relación muchos a muchos resuelta mediante **property_additional_features**.

Una publicación comercial puede tener múltiples fotografías asociadas, mientras que cada fotografía pertenece a una única publicación. Asimismo, una unidad de portafolio puede dar lugar a múltiples publicaciones a lo largo del tiempo, aunque funcionalmente conviene restringir la existencia simultánea de más de una publicación activa por unidad, si así lo exige la regla de negocio.

Cardinalidades en el esquema *landlord_portfolio*

Un usuario puede tener uno o varios portafolios, mientras que cada portafolio pertenece a un único usuario responsable. Un portafolio puede contener múltiples unidades administradas, mientras que cada unidad pertenece a un único portafolio. Una propiedad puede estar asociada a una o varias unidades de portafolio

dependiendo de la interpretación funcional del activo dentro del sistema; si el negocio exige unicidad operativa del inmueble dentro del portafolio, dicha regla debe formalizarse como restricción adicional.

Una unidad de portafolio puede tener múltiples ciclos de arriendo a lo largo del tiempo, pero el modelo debe garantizar que no existan dos ciclos activos superpuestos para la misma unidad. Esta restricción es clave para preservar coherencia temporal en la ocupación del inmueble.

Cardinalidades en el esquema *tracking_process*

Cada lease puede tener múltiples registros históricos de estado, mientras que cada registro histórico corresponde a un único lease y a un único estado del catálogo. Cada lease debe tener, como máximo, un registro de estado actual vigente dentro de **lease_current_status**.

De manera equivalente, cada listing puede tener múltiples registros históricos de estado en **listing_status_history**, pero únicamente un registro vigente en **listing_current_status**. Esta restricción permite separar la trazabilidad histórica de la consulta operativa del estado actual.

Cardinalidades en el esquema *payments*

Cada lease puede tener múltiples obligaciones programadas en **scheduled_payments**, mientras que cada obligación programada pertenece a un único ciclo de arriendo. Cada obligación programada puede estar asociada a uno o varios pagos si el modelo evoluciona hacia escenarios de pago parcial o múltiple; sin embargo, en la versión actual del modelo la relación está planteada de manera directa, donde cada pago referencia una única obligación programada.

Cada pago puede tener múltiples registros de log asociados en **payment_logs**, mientras que cada log pertenece a un único pago. Tanto pagos como logs se vinculan a un único estado del catálogo **payment_status** en un momento dado.

Cardinalidades en el esquema *accounting*

Un portafolio puede tener múltiples reportes agregados generados en distintos cortes o ventanas temporales, mientras que cada reporte agregado corresponde a un único portafolio. De igual forma, una unidad de portafolio puede tener múltiples reportes individuales a lo largo del tiempo, mientras que cada reporte individual corresponde a una única unidad.

Estas cardinalidades reflejan que la reportería es periódica y acumulativa, no una representación única y estática del desempeño financiero.

Cardinalidades en el esquema *users*

Un usuario puede tener uno o varios roles y un rol puede estar asignado a múltiples usuarios, relación resuelta mediante **users_roles**. Del mismo modo, un rol puede contener múltiples permisos y un permiso puede pertenecer a múltiples roles, relación resuelta mediante **roles_permissions**.

Cada usuario puede tener una única extensión de detalle como persona natural o una única extensión como persona jurídica. Esta relación debe gobernarse mediante reglas que eviten que un mismo usuario tenga simultáneamente detalles incompatibles con su naturaleza funcional.

Cardinalidades en el esquema *notifications*

Un usuario puede tener múltiples preferencias de notificación, una por cada combinación relevante de tipo de evento y canal configurado. Cada preferencia pertenece a un único usuario y referencia un único tipo de notificación. Un tipo de notificación puede estar asociado a múltiples preferencias de distintos usuarios.

Cardinalidades en el esquema *contracts*

Cada lease puede tener uno o varios contratos a lo largo del tiempo, mientras que cada contrato pertenece a un único ciclo de arriendo. Cada contrato puede tener múltiples partes asociadas en **contract_party**, y cada parte contractual referencia un único usuario dentro del sistema.

Cada contrato puede tener múltiples archivos documentales y cada archivo pertenece a un único contrato, con un único tipo documental y un único estado documental vigente. Cada parte contractual puede generar uno o

varios eventos de firma, mientras que cada evento de firma pertenece a una única parte contractual y se encuentra asociado a un único estado de firma.

Cada evento de firma puede tener múltiples logs asociados en **signing_logs**, mientras que cada log corresponde a un único evento de firma y a un único estado del catálogo de firma.

Restricciones de unicidad

El modelo debe incorporar restricciones de unicidad que eviten duplicidad en registros cuya identidad funcional no depende únicamente de la clave primaria técnica. Entre ellas se recomiendan restricciones sobre combinaciones como usuario-rol, rol-permiso, propiedad-feature adicional, usuario-tipo de notificación-canal, y aquellas necesarias para impedir más de un estado actual por entidad monitoreada.

Adicionalmente, conviene establecer unicidad lógica para publicaciones activas por unidad de portafolio y para ciclos de arriendo activos por unidad, si tales reglas forman parte del dominio operativo.

Restricciones de obligatoriedad y nulabilidad

Las relaciones esenciales del dominio deben modelarse con obligatoriedad explícita. Por ejemplo, una unidad de portafolio no debe existir sin portafolio ni propiedad asociada; un lease no debe existir sin unidad y arrendatario; una obligación programada no debe existir sin lease; y un pago no debe existir sin obligación programada y estado definido.

Los campos opcionales deben reservarse para información complementaria o dependiente de una etapa posterior del proceso, como ciertos artefactos documentales, logs externos o atributos extensibles.

Representación de historiales y estados actuales

El diseño lógico contempla la coexistencia de estructuras orientadas a registrar historiales completos de eventos y estructuras que representan el estado actual de entidades operativas. Esta dualidad permite soportar tanto necesidades de auditoría y trazabilidad como consultas eficientes orientadas a la operación diaria del sistema.

Separación entre estado actual e historial

Para los procesos que presentan evolución temporal relevante, el modelo define entidades diferenciadas para:

- el registro histórico de eventos o cambios de estado
- la representación del estado vigente en un momento determinado

Esta separación permite mantener un historial completo sin comprometer el rendimiento de consultas operativas frecuentes, que suelen requerir únicamente la condición actual de la entidad.

Historial de estados del ciclo de arriendo

El ciclo de arriendo se modela mediante:

- una entidad de catálogo de estados
- una entidad de historial de estados
- una entidad de estado actual (snapshot operativo)

La entidad de historial registra cada transición ocurrida durante la vida del arriendo, incluyendo la referencia al estado alcanzado y la marca temporal correspondiente. La entidad de estado actual mantiene la condición vigente del lease, evitando la necesidad de calcularla dinámicamente a partir del historial.

Esta estrategia permite reconstruir la trayectoria completa del proceso y, al mismo tiempo, optimizar la lectura del estado operativo vigente.

Historial de estados de publicaciones

La publicación comercial de inmuebles sigue una lógica equivalente a la del ciclo de arriendo. Se dispone de:

- un catálogo de estados de publicación
- un historial de cambios de estado
- una representación del estado actual

Esto permite modelar la evolución comercial de la oferta inmobiliaria, distinguiendo entre publicaciones activas, pausadas, finalizadas u otras condiciones definidas por el dominio.

Consistencia entre historial y estado actual

El modelo asume que el estado actual debe ser coherente con el último evento registrado en el historial correspondiente. Esta consistencia puede garantizarse mediante:

- reglas de integridad lógica en la capa de aplicación
- restricciones o validaciones en la capa de persistencia
- procesos de sincronización que actualicen el snapshot cada vez que se registra un cambio histórico

El objetivo es evitar divergencias entre la representación histórica y la condición operativa vigente.

Diseño de interfaz y experiencia de usuario

Introducción

Propósito del documento

El propósito de este documento es definir los principios, decisiones y lineamientos que orientan el diseño de la interfaz y la experiencia de usuario del sistema, garantizando su alineación con el modelo conceptual del dominio, la arquitectura de software y los objetivos funcionales del producto. Asimismo, busca servir como referencia técnica y conceptual para los equipos de diseño, desarrollo y arquitectura, permitiendo asegurar consistencia en la implementación, facilitar la toma de decisiones y apoyar la evolución controlada de la experiencia digital a lo largo del ciclo de vida del sistema.

Alcance

Este documento abarca la definición del diseño de interacción, la estructura visual de la interfaz, la organización de la información y los criterios de accesibilidad aplicables al sistema, considerando los flujos funcionales principales y la representación de las entidades del dominio en la experiencia de usuario. Incluye además lineamientos para la validación temprana mediante prototipos, la estandarización de componentes de interfaz y las implicaciones técnicas del diseño en la arquitectura de frontend. No contempla el detalle de implementación visual específica ni la definición exhaustiva de activos gráficos finales, los cuales serán desarrollados en etapas posteriores del proyecto.

Diseño

Principios de diseño y accesibilidad

Esta sección establece los fundamentos conceptuales que orientan la experiencia visual y de interacción del sistema. Los principios aquí definidos buscan garantizar coherencia entre la identidad de la plataforma, la comprensión funcional por parte de los usuarios y el cumplimiento de criterios de accesibilidad que permitan una adopción inclusiva y sostenible en el tiempo.

Personalidad de marca

La personalidad de marca define el carácter con el que el sistema se presenta ante sus usuarios y el tipo de relación que busca establecer con ellos. En un contexto asociado a decisiones patrimoniales y contractuales, la experiencia debe proyectar confianza y profesionalismo sin generar distancia o rigidez excesiva.

La marca se caracteriza por ser **formal, sobria, cercana, confiable y fresca**. La plataforma debe percibirse como un entorno corporativo serio, pero contemporáneo, evitando tanto una estética excesivamente exclusiva como una apariencia institucional obsoleta. El tono visual y comunicacional busca transmitir seguridad y claridad a usuarios de distintos grupos etarios.

Objetivo emocional

El objetivo emocional del diseño se centra en generar una experiencia que reduzca la incertidumbre y la carga cognitiva asociada a procesos complejos como la exploración y publicación de inmuebles. La interfaz debe facilitar la comprensión de las acciones, estados y resultados, promoviendo una sensación de control y tranquilidad durante la interacción.

En consecuencia, el diseño busca transmitir **confianza, claridad y calma**, mediante interacciones predecibles, jerarquías visuales consistentes y un lenguaje visual legible. La reducción de fricción en la navegación y en la ejecución de tareas críticas constituye un criterio transversal para la definición de flujos, componentes y patrones de interacción.

Accesibilidad (WCAG)

La accesibilidad se aborda como un requisito fundamental del sistema, por tanto, se debe incorporar desde las primeras decisiones de diseño para garantizar que la plataforma sea usable y comprensible para una amplia diversidad de usuarios, independientemente de sus capacidades o condiciones de acceso.

El nivel objetivo actual corresponde al cumplimiento de **WCAG 2.1 AA**, con la posibilidad de endurecer los criterios hacia **AAA** mediante ajustes menores, como el incremento de contraste o el aumento de tamaños tipográficos. Entre las reglas base se establece un contraste mínimo de 4.5:1 para texto normal y de 3:1 para texto grande (≥ 18 px o ≥ 14 px en negrita). Asimismo, no se debe utilizar el color como único medio para comunicar estados del sistema, debiendo complementarse con íconos, texto o patrones visuales. Finalmente, los elementos interactivos deben contar con un área táctil mínima de 44 x 44 px en dispositivos móviles, garantizando así condiciones adecuadas de interacción.

Arquitectura de Interacción y Navegación

Esta sección define cómo la experiencia de usuario materializa la arquitectura lógica del sistema, asegurando que la organización de la navegación, los flujos de interacción y la presentación de estados reflejen de manera coherente la estructura del dominio y las responsabilidades de cada componente tecnológico. La arquitectura de interacción no se concibe como un diseño superficial de pantallas, sino como un mecanismo para traducir la lógica funcional del sistema en recorridos comprensibles, consistentes y predecibles para el usuario. En consecuencia, las decisiones de navegación y organización de la información deben derivarse de los agregados del dominio, los casos de uso prioritarios y las restricciones técnicas establecidas en la arquitectura de software.

Modelo de navegación basado en el dominio

El modelo de navegación se estructura a partir de las entidades principales del sistema y de los procesos que los usuarios deben ejecutar sobre ellas. Esto implica que la jerarquía de menús, rutas y vistas responda a la lógica conceptual del dominio, evitando configuraciones arbitrarias o basadas únicamente en criterios visuales. En este enfoque, cada sección de la interfaz representa un conjunto de capacidades funcionales relacionadas con un agregado o subdominio específico, facilitando la comprensión del sistema y reduciendo la carga cognitiva del usuario. La navegación debe permitir identificar con claridad el contexto actual, la ubicación dentro del sistema y las posibles acciones disponibles, garantizando continuidad conceptual entre las distintas áreas funcionales.

Flujos funcionales y estados de interfaz

Los flujos de interacción representan la secuencia de acciones que el usuario debe ejecutar para completar procesos definidos en el sistema. Cada flujo debe estar alineado con las transiciones de estado válidas de las entidades del dominio, asegurando que la experiencia refleje fielmente la lógica operativa del backend. La interfaz debe comunicar de manera explícita el estado actual del proceso, el progreso alcanzado y las condiciones necesarias para avanzar. Esto se logra mediante indicadores visuales, mensajes contextuales y estructuras de navegación que eviten ambigüedad o interpretaciones erróneas sobre la validez de las acciones realizadas. La correcta representación de estados contribuye a la percepción de control y confiabilidad del sistema.

Interacción en procesos multietapa

Los procesos complejos, como la creación o gestión de información estructurada, requieren dividirse en etapas que correspondan a agrupaciones lógicas de datos o responsabilidades funcionales. Esta fragmentación permite reducir la complejidad percibida por el usuario y facilitar la validación progresiva de la información ingresada.

La arquitectura de interacción debe garantizar continuidad entre etapas, manteniendo contexto, consistencia visual y claridad en los objetivos de cada paso. Asimismo, debe contemplar la posibilidad de revisar o modificar información previamente ingresada sin comprometer la integridad del proceso. La división en etapas debe responder a la semántica del dominio y no únicamente a criterios de distribución visual.

Estrategias de retroalimentación del sistema

La retroalimentación del sistema constituye un mecanismo esencial para comunicar al usuario el resultado de sus acciones y el estado de los datos procesados. Esta retroalimentación debe ser oportuna, contextual y coherente con la persistencia real de la información en los servicios backend.

Se deben implementar mecanismos de feedback tanto sincrónicos como asincrónicos, considerando escenarios en los que las operaciones puedan requerir procesamiento diferido o validaciones externas. Los mensajes de confirmación, advertencia o error deben integrarse de manera natural en el flujo de interacción, evitando interrupciones innecesarias o ambigüedades sobre la efectividad de las acciones realizadas.

Manejo de errores y resiliencia

El manejo de errores se concibe como parte integral de la experiencia, no como una situación excepcional. La arquitectura de interacción debe prever escenarios de fallo y proporcionar mecanismos claros para la recuperación, garantizando que el usuario pueda continuar su proceso sin pérdida de información ni necesidad de reiniciar tareas completas.

Los mensajes de error deben explicar de forma comprensible la causa del problema y las acciones necesarias para resolverlo, evitando terminología técnica o ambigua. Además, la interfaz debe contemplar estados intermedios que reflejen situaciones de validación, espera o inconsistencias temporales, contribuyendo a una experiencia resiliente que mantenga la confianza del usuario incluso ante eventos no previstos.

Sistema Visual, Tipografía y Layout

Sistema de color

El sistema de color se define como un mecanismo semántico que permite comunicar prioridades funcionales, estados operativos y jerarquías de interacción dentro de la interfaz. Su aplicación responde a una lógica estructurada que busca reforzar la comprensión del sistema, reducir la ambigüedad perceptual y mantener coherencia transversal entre módulos. La selección cromática se fundamenta en principios de accesibilidad, consistencia visual y representación clara del estado de las entidades del dominio.

Paleta neutral

La paleta neutral constituye la base estructural del sistema visual, proporcionando un entorno visual sobrio que permite destacar el contenido relevante y reducir el ruido perceptual. Estos tonos se emplean principalmente en fondos, divisores y tipografía, contribuyendo a la legibilidad y al orden visual de la interfaz.

Neutrales

	Neutral 50 #F9FAFB Fondo principal de la app
	Neutral 100 #F3F4F6 Fondo secundario, áreas deshabilitadas
	Neutral 300 #D1D5DB Bordes y divisores
	Neutral 600 #4B5563 Texto secundario
	Neutral 900 #111827 Texto principal, encabezados

Color primario (marca y acciones principales)

El color primario representa la identidad operativa del sistema y se utiliza para destacar acciones críticas, estados activos y elementos interactivos de alta prioridad. Su uso consistente permite orientar la atención del usuario hacia los puntos de toma de decisión dentro de cada flujo funcional.

Primary (Azul)

	Primary 50 #EFF6FF Fondo de elementos hover primarios
	Primary 500 #1D4ED8 Botones principales, links, íconos activos
	Primary 600 #1E40AF Hover estado de botones primarios
	Primary 700 #1E3A8A Estado pressed de acciones principales

Color secundario

El color secundario cumple una función de acento visual, permitiendo resaltar elementos informativos o de apoyo sin competir con el color primario. Su aplicación se restringe a componentes de baja criticidad funcional, como indicadores visuales o elementos gráficos complementarios.

Secondary (Verde)

	Secondary 500 #10B981 Acentos, badges de éxito, estado disponible
---	--

Colores de estado

Los colores de estado comunican el resultado de las acciones del usuario y el estado operativo del sistema. Su uso debe ser consistente y acompañado de otros indicadores visuales, garantizando comprensión universal y cumplimiento de criterios de accesibilidad.

Estados	
	Success 500 #10B981 Mensajes de éxito, confirmaciones
	Warning 500 #F59E0B Alertas, estados pendientes
	Error 500 #EF4444 Errores, validaciones fallidas
	Info 500 #3B82F6 Información, tooltips, ayuda

Tipografía y jerarquía

La tipografía constituye un componente estructural del sistema visual, permitiendo establecer jerarquías informativas, mejorar la legibilidad y facilitar la comprensión de los flujos funcionales. La definición de escalas tipográficas responde a un enfoque mobile-first, asegurando adaptabilidad a distintos contextos de uso y dispositivos.

Fuente base

La selección tipográfica prioriza familias modernas, legibles y ampliamente soportadas en entornos digitales. El sistema debe garantizar disponibilidad de al menos tres pesos tipográficos, permitiendo construir jerarquías claras sin recurrir a recursos visuales adicionales.

Ejemplo de familias recomendadas: Inter, Roboto o System UI.

Pesos mínimos requeridos: Regular, Medium y Semibold.

Escala tipográfica mobile-first

La escala tipográfica define el tamaño y peso de los textos según su rol dentro de la interfaz, permitiendo diferenciar títulos, subtítulos y contenido principal. Esta jerarquía contribuye a orientar la atención del usuario y a estructurar la información de forma comprensible.

2. Tipografía	
Fuente principal: Inter	
H1 - Heading 1 Line height: 40px	32px / 700 (Bold)
Encuentra tu hogar ideal	
H2 - Heading 2 Line height: 32px	24px / 700 (Bold)
Propiedades destacadas	
H3 - Heading 3 Line height: 28px	20px / 600 (Semibold)
Apartamento en Zona Norte	
Body 1 Line height: 24px	16px / 400 (Regular)
Este espacioso apartamento cuenta con 3 habitaciones, 2 baños completos y una amplia terraza con vista panorámica.	
Body 2 Line height: 20px	14px / 400 (Regular)
Descripción secundaria o texto de apoyo para tarjetas de contenido y elementos de lista.	
Caption Line height: 16px	12px / 400 (Regular)
Actualizado hace 2 horas • ID: #12345	

Layout y espaciado

El layout define la organización espacial de los componentes dentro de la interfaz, permitiendo estructurar la información de manera clara y coherente con los flujos funcionales del sistema. Un sistema de espaciado consistente facilita la comprensión visual, mejora la legibilidad y contribuye a una experiencia más predecible.

Enfoque mobile-first

El diseño adopta un enfoque mobile-first, priorizando la claridad funcional en pantallas de menor tamaño y extendiendo progresivamente la interfaz hacia dispositivos de mayor resolución. Este enfoque garantiza continuidad de experiencia y optimiza la interacción en contextos de uso móvil.

Uso de una sola columna principal.

- Alineación del contenido a una grilla de espaciado de 4 u 8 pt.
- Márgenes laterales aproximados de 16 a 20 px.

Patrones de layout

Los patrones de layout establecen configuraciones recurrentes de organización de contenido según el tipo de pantalla o flujo funcional. Estos patrones permiten mantener consistencia visual y reducir el esfuerzo cognitivo del usuario al interactuar con distintas áreas del sistema.

Pantallas de lista

- Encabezado con título y acción secundaria (ej. filtros).
- Lista de tarjetas con separación uniforme.
- CTA principal fija cuando sea necesario.

Pantallas de formulario por pasos

- Encabezado con título del flujo.
- Indicador de progreso multietapa.
- Agrupación temática de campos.
- Botón primario al final de la pantalla.

Sistema de espaciado

El sistema de espaciado establece reglas para la separación entre elementos, contribuyendo a una interfaz equilibrada y legible. Estas reglas deben aplicarse de manera uniforme para mantener coherencia visual en toda la plataforma.

- Entre secciones principales: 24–32 px.
- Entre campos relacionados: 8–16 px.
- Padding interno de contenedores: 16–20 px.

Sistema de Componentes y Consistencia Operativa

Componentes como abstracciones del dominio

Los componentes de interfaz deben concebirse como representaciones visuales de unidades funcionales del dominio, reflejando acciones, estados o entidades relevantes dentro del sistema. Esta aproximación permite mantener una correspondencia clara entre la lógica de negocio y la experiencia de usuario, evitando discrepancias conceptuales que puedan generar confusión o errores operativos.

Por ejemplo, componentes como tarjetas, formularios o indicadores de estado no solo organizan información, sino que comunican la naturaleza y el estado de las entidades subyacentes. De esta manera, el diseño de componentes facilita la trazabilidad entre decisiones de interacción y comportamiento del sistema, contribuyendo a una experiencia coherente y comprensible.

Estados interactivos y comportamiento consistente

Cada componente debe contemplar un conjunto definido de estados interactivos que reflejen con precisión las condiciones operativas del sistema. Estados como reposo, foco, selección, validación o error deben comunicarse de manera consistente en toda la interfaz, permitiendo al usuario interpretar correctamente el resultado de sus acciones.

La consistencia en el comportamiento interactivo reduce la incertidumbre perceptual y favorece la predictibilidad del sistema. Esto implica definir reglas claras para la transición entre estados, así como

garantizar que dichas transiciones estén alineadas con la lógica de procesamiento de datos y las respuestas del backend.

Reutilización y evolución controlada

La reutilización de componentes constituye un principio clave para la eficiencia del desarrollo y la sostenibilidad del diseño. Al centralizar la definición de patrones visuales y funcionales, es posible introducir mejoras o ajustes de manera transversal sin afectar la coherencia global del sistema.

La evolución del sistema de componentes debe gestionarse como un proceso arquitectónico, en el cual cada modificación sea evaluada en función de su impacto en la experiencia de usuario, la mantenibilidad del código y la consistencia del producto. Esto permite asegurar que el crecimiento funcional del sistema no derive en fragmentación visual o técnica.

Integración con la arquitectura frontend

Los componentes deben diseñarse como unidades desacopladas que puedan integrarse de forma modular en la arquitectura de frontend. Esta aproximación facilita la separación de responsabilidades entre la lógica de presentación, la gestión de estado y la comunicación con servicios backend, promoviendo prácticas de desarrollo escalables y mantenibles.

La definición clara de interfaces, propiedades y comportamientos esperados para cada componente permite su implementación en frameworks modernos, así como su reutilización en distintos contextos funcionales sin necesidad de redefinir su lógica base. De esta manera, el sistema de componentes contribuye a la estabilidad técnica y a la consistencia operativa del frontend.

Prototipos, Validación Temprana y Evolución de la Experiencia

Esta sección describe el enfoque adoptado para validar tempranamente el diseño de la experiencia de usuario antes de su implementación definitiva, con el propósito de reducir riesgos de usabilidad, detectar inconsistencias entre la interfaz y el dominio, y orientar decisiones arquitectónicas de manera informada.

Rol de los prototipos en la arquitectura UX

Los prototipos constituyen herramientas estratégicas para explorar, validar y comunicar decisiones de diseño de experiencia en contextos donde la complejidad funcional o la sensibilidad del dominio requieren un alto grado de comprensión por parte del usuario. Su uso permite evaluar la correspondencia entre los modelos mentales de los usuarios y la lógica conceptual del sistema, facilitando la identificación de posibles fricciones antes de la fase de desarrollo.

Asimismo, los prototipos contribuyen a reducir el retrabajo en la implementación frontend, al permitir validar anticipadamente flujos críticos, estructuras de navegación y patrones de interacción. Desde una perspectiva arquitectónica, su utilización favorece la alineación entre diseño, desarrollo y objetivos del producto, promoviendo decisiones más robustas y coherentes.

Enfoque metodológico de prototipado

Para la construcción del prototipo se seleccionó la herramienta Figma, debido a su madurez en el ecosistema de diseño UX/UI, su capacidad para estructurar sistemas de diseño reutilizables y su integración con funcionalidades asistidas por inteligencia artificial.

El proceso inició con la definición de la guía de diseño del sistema, estableciendo previamente los tokens visuales asociados a color, tipografía, jerarquía y layout. Esta etapa permitió consolidar un lenguaje visual coherente que posteriormente serviría como base para la generación automatizada de interfaces.

En una fase posterior, se incorporó el uso de herramientas de inteligencia artificial como soporte al diseño, integrando modelos de lenguaje en la definición de prompts especializados para la generación de prototipos.

Esta aproximación permitió acelerar la exploración de alternativas de interacción y reducir el esfuerzo manual en la construcción inicial de las pantallas.

Integración de inteligencia artificial en el diseño de interfaces

El proceso de prototipado incluyó el uso combinado de un modelo de lenguaje para la generación de instrucciones de diseño y la funcionalidad Figma Make para la construcción visual de las interfaces. Inicialmente, se proporcionó al modelo de lenguaje el contexto completo del sistema, incluyendo requerimientos funcionales, decisiones arquitectónicas y el modelo conceptual de datos. Con base en esta información, se definieron los flujos funcionales que debían representarse en el prototipo, tales como exploración de inmuebles, gestión de arriendos, visualización de contratos y administración de perfiles de usuario.

Para cada flujo funcional se solicitó la generación de prompts optimizados que posteriormente fueron utilizados como insumo en Figma Make, junto con el sistema de diseño previamente definido. Esta integración permitió construir de forma iterativa las diferentes rutas de navegación y estados de interacción del sistema. El proceso completo de diseño, implementación visual y ajustes implicó un total de treinta y cinco iteraciones durante un periodo de dos días, lo cual permitió alcanzar una primera versión navegable del prototipo.

Construcción del prototipo navegable

El resultado del proceso iterativo fue la construcción de un prototipo visual con múltiples rutas y flujos funcionales implementados, representando los principales escenarios de interacción previstos para el sistema. Figma permitió la publicación del prototipo en un entorno accesible vía internet, facilitando su distribución y evaluación por parte de usuarios potenciales sin necesidad de implementación tecnológica real. Esta capacidad resultó fundamental para obtener retroalimentación temprana sobre la experiencia de uso, la comprensión de los procesos y la percepción general de la interfaz.

Las evidencias visuales del prototipo, incluyendo las principales pantallas y flujos navegables, se presentan en esta sección como soporte documental de las decisiones de diseño adoptadas.

Evidencias visuales del diseño propuesto

Funcionalidades mixtas

Estas funcionalidades se pueden ver en un usuario anónimo, un arrendador y un arrendatario.

Explorar inmuebles

Filtros

Más recientes



Casa · Usaquén

\$4,200,000

Publicado hace 1 día

3

2



Dúplex · Suba

\$5,200,000

Publicado hace 2 días

4

3



Apartamento · Chapinero

\$2,500,000

Publicado hace 3 días

2

1



Filtros

Ciudad

Zona / Barrio

Primero selecciona una ciudad

Fecha de publicación

Tipo de propiedad

Precio mensual

Número de habitaciones

Número de baños

Área (m²)

Aplicar filtros

Limpiar filtros



Detalle del inmueble



\$3,000,000/mes

Inmueble en Bogotá



2

Habitaciones



2

Baños



70 m²

Área

Descripción

Excelente inmueble en zona privilegiada con todas las comodidades.

Características

Parqueadero

Seguridad 24/7

Ubicación

 Bogotá, Colombia

Contactar arrendador

Funcionalidades del arrendador

A continuación se presentan las funcionalidades exclusivas del arrendador:

Mis contratos

Apartamento 2 hab.

Carrera 7 #58-32, Chapinero

En firma

Arrendatario

María González

Canon mensual

\$2,500,000

Casa 3 hab.

Calle 85 #15-25, Usaquén

Vigente

Arrendatario

Carlos Pérez

Canon mensual

\$4,200,000

15 de febrero, 2026 - 15 de febrero, 2027

Studio

Carrera 5 #12-10, La Candelaria

Vigente

Arrendatario

Ana Martínez

Canon mensual

\$1,800,000

1 de enero, 2026 - 1 de enero, 2027

Penthouse

Carrera 9 #75-20, Rosales

Vigente

Arrendatario

Luis Rodríguez

Canon mensual

\$8,500,000

1 de marzo, 2026 - 1 de marzo, 2027

Apartamento 3 hab.

Transversal 24 #98-10, Chicó

Borrador

Arrendatario

Patricia Silva

Canon mensual

\$3,800,000

Local Comercial

Avenida 19 #120-40, Santa Bárbara

Vigente

Arrendatario

Roberto Vargas

Canon mensual

\$6,500,000

1 de diciembre, 2025 - 1 de diciembre, 2026

Mostrando 1 a 6 de 8 resultados

Items por página: 5

<

1

2

>

Contrato de arriendo

En firma

Inmueble

Apartamento 2 hab.

Carrera 7 #58-32, Chapinero

Partes del contrato

Arrendador

Juan Pérez

Arrendatario

María González

Términos del contrato

Fecha de inicio

1 de abril, 2026

Fecha de finalización

1 de abril, 2027

Canon mensual

\$2,500,000

Depósito

\$2,500,000

Funcionalidades del arrendatario

Ahora se presenta las funcionalidades exclusivas del arrendatario:

Menú



María González
Arrendatario



Explorar inmuebles



Mi arriendo



Notificaciones



Mis contratos



Mi perfil



Cerrar sesión



Mi arriendo



Apartamento

Chapinero
Carrera 7 #58-32

Canon mensual

\$2,500,000

Próximo pago

Pendiente



Fecha de vencimiento
15 de Abril, 2026



Valor a pagar
\$2,500,000

Pagar ahora

Ver historial de pagos >

←

Historial de pagos

Todos

Pendientes

Pagados

Vencidos

Abril 2026

Vence: 15 de Abril

Pendiente

\$2,500,000

Pagar >

Marzo 2026

Vence: 15 de Marzo

Pagado

\$2,500,000

Ver comprobante

Febrero 2026

Vence: 15 de Febrero

Pagado

\$2,500,000

Ver comprobante

Enero 2026

Vence: 15 de Enero

Pagado

\$2,500,000

Ver comprobante

←

Detalle de pago

Resumen de la cuota

Abril 2026

Canon de arrendamiento

\$2,500,000

Administración

\$150,000

Servicios públicos

\$80,000

Total a pagar

\$2,730,000

Fecha de vencimiento: 15 de Abril, 2026

Evita cargos adicionales pagando antes de esta fecha

Selecciona un medio de pago

Tarjeta débito/crédito

Paga de forma rápida y segura con tu tarjeta

>

Transferencia bancaria / PSE

Realiza el pago desde tu banco de confianza

>

Notificaciones

Contratos pendientes

Contrato pendiente de firma

Acción requerida

Apartamento en Chapinero - Carrera 7 #58-32

21 de marzo, 2026

Revisar y firmar

Contrato firmado exitosamente

Studio en La Candelaria

15 de marzo, 2026

Contrato pendiente de firma

Acción requerida

Casa en Usaquén - Calle 85 #15-25

18 de marzo, 2026

Revisar y firmar

Contrato firmado exitosamente

Apartamento en Rosales

10 de marzo, 2026

Contrato firmado exitosamente

Local Comercial en Santa Bárbara

5 de marzo, 2026

Recordatorio de pago

Acción requerida

Pago mensual vence en 3 días

22 de marzo, 2026

Revisar y firmar

Mostrando 1 a 6 de 7 resultados

Items por página: 5

<

1

2

>

Validación con usuarios potenciales

Una vez publicado el prototipo, se compartió el enlace de acceso con usuarios representativos de los roles definidos en el sistema, específicamente arrendadores y arrendatarios. Esta validación tuvo como objetivo identificar oportunidades de mejora en la experiencia de interacción, evaluar la comprensión de los flujos funcionales y analizar la percepción general del producto digital.

Las observaciones recibidas se centraron principalmente en:

- la expectativa de funcionalidades presentes en plataformas existentes que no estaban contempladas inicialmente, como la posibilidad de visualizar las imágenes de los inmuebles en modo ampliado al seleccionarlasy desde la vista detallada.
- la necesidad de incorporar filtros adicionales, por ejemplo, la capacidad de filtrar inmuebles según la disponibilidad de parqueadero.
- la confusión generada por ciertas iteraciones de diseño en campos específicos, lo que evidenció la necesidad de mejorar la claridad en la organización de la información.
- la percepción de tamaños tipográficos excesivos por parte de usuarios jóvenes, aspecto que se encontraba alineado con la estrategia de accesibilidad orientada a adultos mayores.
- la valoración positiva de la simplicidad visual, la baja carga cognitiva y la percepción de frescura derivada del uso del sistema de color.

Iteración y consolidación del prototipo hacia la versión MVP

Las observaciones obtenidas durante la validación fueron analizadas en función de su alineación con los objetivos del sistema y el alcance definido para la versión mínima viable (MVP). Aquellas sugerencias consideradas coherentes con la estrategia funcional del producto fueron incorporadas mediante iteraciones adicionales del diseño.

Este proceso implicó quince iteraciones adicionales, desarrolladas en un periodo de un día, alcanzando un total de cincuenta iteraciones de diseño. Como resultado, se consolidó un prototipo más refinado, con mayor claridad en los flujos de interacción, mejor alineación con las expectativas de los usuarios y una representación más precisa del modelo conceptual del sistema.

La iteración controlada del prototipo permitió reducir riesgos asociados a la implementación posterior, mejorar la calidad de la experiencia proyectada y fortalecer la coherencia entre diseño, arquitectura y requerimientos funcionales.